

finalproject - implementation

June 28, 2026

1 Final project implementation - F1 clustering project - Oren Yehezkel, Ido Brener and Roe Amsalem

1.1 Abstract

This project investigates the distinct driving styles of Formula 1 drivers by applying unsupervised machine learning techniques to high-resolution telemetry data. Using the OpenF1 API, we extracted spatial and telemetry data (speed, throttle, braking, RPM) from the fastest qualifying laps of the 2023 season across four diverse circuits (Monza, Spa, Singapore, Suzuka). The primary objective is to determine whether drivers can be clustered into meaningful profiles (e.g., Aggressive, Balanced, Conservative) based on their cornering behavior.

The methodology involves dynamic corner detection, extensive feature engineering (calculating apex speeds, trail braking percentages, and throttle application), and dimensionality reduction using Principal Component Analysis (PCA). We evaluated K-Means clustering to partition the drivers, validating the results using Silhouette scores and t-SNE visualizations. The primary findings reveal distinct clustering among drivers, highlighting how different approaches to corner entry and exit translate into overall driving profiles, providing data-driven insights into the mechanics of elite motorsport.

1.2 Introduction

Formula 1 is a highly data-driven sport where milliseconds separate the winner from the rest of the pack. While car performance is a significant factor, individual driving style plays a crucial role in race strategy, tire management, and overall lap time. The application domain of this project is sports analytics and vehicle telemetry.

The main question we are investigating is: **Can we mathematically group F1 drivers into distinct driving styles based solely on their telemetry data during high-speed corners?**

This dataset is highly interesting because it provides raw, unlabelled, high-frequency sensor data, representing human decision-making at the limit of physical grip. Clustering is the ideal tool for this investigation because there are no predefined “labels” for driving styles. By grouping drivers based on their multidimensional telemetry features, we can discover hidden patterns and objectively define what constitutes an aggressive or conservative driver without human bias.

2 1. Environment Setup & Library Imports

2.0.1 Importing the necessary Python libraries for data manipulation, API requests, visualization, and machine learning models (PCA, K-Means, etc.).

```
[2]: import requests
import pandas as pd
import os
import time
import matplotlib.pyplot as plt
import seaborn as sns
import os
import numpy as np
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.manifold import TSNE
from matplotlib.pyplot import savefig
```

2.1 Dataset Description

Our dataset is derived from the OpenF1 API, focusing on the 2023 Qualifying sessions across four distinct tracks: Monza, Spa, Singapore, and Suzuka. * **Samples:** Each sample originally represents a single telemetry ping during a driver's fastest qualifying lap. After feature engineering, each sample represents a driver's specific behavior through a single technical corner. * **Features:** The raw features include X and Y spatial coordinates, speed, brake (boolean), throttle percentage, RPM, and gear. Additionally, we manually integrated demographic data (age, experience, team) to contextualize the clusters. * **Size:** The raw dataset contains hundreds of thousands of telemetry points. After filtering and aggregating per corner, the dataset is reduced to a concise matrix of core cornering events per driver. * **Limitations:** The primary limitations include API rate limits, missing sensor data for certain drivers, and the challenge of aligning GPS coordinates with car telemetry (which operate at slightly different frequencies).

3 Data Acquisition from OpenF1 API

3.0.1 Functions to fetch telemetry and location data for the 2023 Qualifying sessions. A retry mechanism is implemented to handle API rate limits and connection timeouts safely.

```
[3]: # Function to fetch the exact list of drivers that participated in a specific
    ↪ session
# Meeting key and session key are required to query the OpenF1 API

# Smart fetch function with an automatic retry mechanism to prevent crashes due
    ↪ to server overload (HTTP 429)
# Sleeps for 2 seconds before retrying
```

```

# Main function to initiate the data ingestion process for Monza, Spa,
↳Singapore, and Suzuka
# Creates a dedicated folder for raw data and saves the output as CSV files
def get_drivers_for_session(meeting_id, session_id):
    """Fetches the exact list of drivers that participated in a specific
    ↳session."""
    url = f"https://api.openf1.org/v1/sessions?
    ↳meeting_key={meeting_id}&session_key={session_id}"
    try:
        res = requests.get(url, timeout=30)
        if res.status_code == 200 and len(res.json()) > 0:
            drivers_url = f"https://api.openf1.org/v1/drivers?
            ↳meeting_key={meeting_id}&session_key={session_id}"
            d_res = requests.get(drivers_url, timeout=30)
            if d_res.status_code == 200:
                return [d['driver_number'] for d in d_res.json()]
    except Exception as e:
        print(f"    [!] Failed to fetch drivers for session {session_id}: {e}")
    return []

def fetch_with_retry(url, max_retries=3):
    """Attempts to fetch data from a URL with automatic retries on failure."""
    for attempt in range(max_retries):
        try:
            res = requests.get(url, timeout=45)
            if res.status_code == 200:
                return res.json()
            elif res.status_code == 429: # Too many requests limit
                print(f"    [Rate Limit] Sleeping for 2s... (Attempt
                ↳{attempt+1}/{max_retries})")
                time.sleep(2)
            else:
                return []
        except Exception as e:
            if attempt < max_retries - 1:
                time.sleep(2)
            else:
                print(f"\n    [!] Permanent timeout after {max_retries} retries
                ↳for URL: {url}")
                return []
    return []

def download_full_track_data():
    # Set the new target directory for Qualifying data
    base_dir = "raw_data_qualifying"
    os.makedirs(base_dir, exist_ok=True)

```

```

# 2023 Qualifying Sessions identifiers
track_keys = {
    'Monza': {'meeting_key': 1218, 'session_key': 9153},
    'Spa': {'meeting_key': 1216, 'session_key': 9135},
    'Singapore': {'meeting_key': 1219, 'session_key': 9161},
    'Suzuka': {'meeting_key': 1220, 'session_key': 9169}
}

print(f"--- Starting Data Ingestion into '{base_dir}' folder ---")

for track_name, keys in track_keys.items():
    session_id = keys['session_key']
    meeting_id = keys['meeting_key']
    print(f"\n[Processing] Track: {track_name} | Meeting: {meeting_id} |_
↳Session: {session_id}")

    # Define expected file paths in the new directory
    loc_path = os.path.join(base_dir, f"{track_name}_raw_location.csv")
    car_path = os.path.join(base_dir, f"{track_name}_raw_telemetry.csv")
    laps_path = os.path.join(base_dir, f"{track_name}_raw_laps.csv")

    need_loc = not os.path.exists(loc_path)
    need_car = not os.path.exists(car_path)
    need_laps = not os.path.exists(laps_path)

    if not need_loc and not need_car and not need_laps:
        print(f" [SKIPPED] All files for {track_name} already exist in_
↳{base_dir}.")
        continue

    # Dynamically fetch the drivers who actually participated
    session_drivers = get_drivers_for_session(meeting_id, session_id)
    if not session_drivers:
        print(f" [!] Could not fetch dynamic drivers. Falling back to_
↳default list.")
        session_drivers = [1, 2, 3, 4, 10, 11, 14, 16, 18, 20, 22, 23, 24,_
↳27, 31, 40, 44, 55, 63, 77, 81]

    print(f" -> Found {len(session_drivers)} drivers for this session.")

    track_location_data = []
    track_car_data = []
    track_laps_data = []

    for driver in session_drivers:
        print(f" -> Fetching data for Driver {driver}...", end="\r")

```

```

# 1. Location Data
if need_loc:
    loc_url = f"https://api.openf1.org/v1/location?
↪meeting_key={meeting_id}&session_key={session_id}&driver_number={driver}"
    data = fetch_with_retry(loc_url)
    if data: track_location_data.append(pd.DataFrame(data))

# 2. Telemetry Data
if need_car:
    car_url = f"https://api.openf1.org/v1/car_data?
↪meeting_key={meeting_id}&session_key={session_id}&driver_number={driver}"
    data = fetch_with_retry(car_url)
    if data: track_car_data.append(pd.DataFrame(data))

# 3. Laps Data
if need_laps:
    laps_url = f"https://api.openf1.org/v1/laps?
↪meeting_key={meeting_id}&session_key={session_id}&driver_number={driver}"
    data = fetch_with_retry(laps_url)
    if data: track_laps_data.append(pd.DataFrame(data))

# Brief pause to respect API limits
time.sleep(0.5)

print(f" -> Download complete for {track_name}! Merging files...      ↵
↪")

# Save merged files into the new directory
if need_loc and track_location_data:
    pd.concat(track_location_data, ignore_index=True).to_csv(loc_path,↵
↪index=False)
    print(f" [SUCCESS] Location data saved.")

if need_car and track_car_data:
    pd.concat(track_car_data, ignore_index=True).to_csv(car_path,↵
↪index=False)
    print(f" [SUCCESS] Telemetry data saved.")

if need_laps and track_laps_data:
    pd.concat(track_laps_data, ignore_index=True).to_csv(laps_path,↵
↪index=False)
    print(f" [SUCCESS] Laps data saved.")

print("\n--- All Data Downloaded & Saved Successfully to↵
↪'raw_data_qualifying' ---")

```

```
#download_full_track_data()
```

4 Importing driver-related data

```
[4]: # dictionary containing all the drivers non-race data
drivers_data = [
    {
        'driver_number':1, 'name':'Max Verstappen', 'team':'Red Bull Racing',
        'age_in_2023':26, 'years_of_experience':9, 'years_in_current_team':
↪8, 'contract_years_remaining':5,
        'country':'Netherlands', 'marital_status':'in a relationship',
↪'has_kids':False,
        'world_championships':2, 'height_cm':181, 'weight_kg':
↪72, 'driver_status':1
    },
    {
        'driver_number': 11, 'name': 'Sergio Perez', 'team': 'Red Bull Racing',
        'age_in_2023': 33, 'years_of_experience': 13, 'years_in_current_team':
↪3, 'contract_years_remaining':1,
        'country': 'Mexico', 'marital_status': 'Married', 'has_kids': True,
        'world_championships': 0, 'height_cm': 173, 'weight_kg': 63,
↪'driver_status': 2
    },
    {
        'driver_number': 44, 'name': 'Lewis Hamilton', 'team': 'Mercedes',
        'age_in_2023': 38, 'years_of_experience': 17, 'years_in_current_team':
↪11, 'contract_years_remaining':0,
        'country': 'United Kingdom', 'marital_status': 'Single', 'has_kids':
↪False,
        'world_championships': 7, 'height_cm': 174, 'weight_kg': 73,
↪'driver_status': 1
    },
    {
        'driver_number': 63, 'name': 'George Russell', 'team': 'Mercedes',
        'age_in_2023': 25, 'years_of_experience': 5, 'years_in_current_team':
↪2, 'contract_years_remaining':2,
        'country': 'United Kingdom', 'marital_status': 'In a Relationship',
↪'has_kids': False,
        'world_championships': 0, 'height_cm': 185, 'weight_kg': 70,
↪'driver_status': 2
    },
    {
        'driver_number': 16, 'name': 'Charles Leclerc', 'team': 'Ferrari',
```

```

        'age_in_2023': 26, 'years_of_experience': 6, 'years_in_current_team': 5,
        ↪ 'contract_years_remaining': 1,
        'country': 'Monaco', 'marital_status': 'In a Relationship', 'has_kids':
        ↪ False,
        'world_championships': 0, 'height_cm': 180, 'weight_kg': 69,
        ↪ 'driver_status': 1
    },
    {
        'driver_number': 55, 'name': 'Carlos Sainz', 'team': 'Ferrari',
        'age_in_2023': 29, 'years_of_experience': 9, 'years_in_current_team':
        ↪ 3, 'contract_years_remaining': 1,
        'country': 'Spain', 'marital_status': 'In a Relationship', 'has_kids':
        ↪ False,
        'world_championships': 0, 'height_cm': 178, 'weight_kg': 66,
        ↪ 'driver_status': 2
    },
    {
        'driver_number': 4, 'name': 'Lando Norris', 'team': 'McLaren',
        'age_in_2023': 24, 'years_of_experience': 5, 'years_in_current_team':
        ↪ 5, 'contract_years_remaining': 2,
        'country': 'United Kingdom', 'marital_status': 'Single', 'has_kids':
        ↪ False,
        'world_championships': 0, 'height_cm': 170, 'weight_kg': 68,
        ↪ 'driver_status': 1
    },
    {
        'driver_number': 81, 'name': 'Oscar Piastri', 'team': 'McLaren',
        'age_in_2023': 22, 'years_of_experience': 1, 'years_in_current_team':
        ↪ 1, 'contract_years_remaining': 3,
        'country': 'Australia', 'marital_status': 'In a Relationship',
        ↪ 'has_kids': False,
        'world_championships': 0, 'height_cm': 178, 'weight_kg': 68,
        ↪ 'driver_status': 2
    },
    {
        'driver_number': 14, 'name': 'Fernando Alonso', 'team': 'Aston Martin',
        'age_in_2023': 42, 'years_of_experience': 20, 'years_in_current_team':
        ↪ 1, 'contract_years_remaining': 1,
        'country': 'Spain', 'marital_status': 'In a Relationship', 'has_kids':
        ↪ False,
        'world_championships': 2, 'height_cm': 171, 'weight_kg': 68,
        ↪ 'driver_status': 1
    },
    {
        'driver_number': 18, 'name': 'Lance Stroll', 'team': 'Aston Martin',

```

```

        'age_in_2023': 25, 'years_of_experience': 7, 'years_in_current_team': 0,
        ↪5, 'contract_years_remaining': 3,
        'country': 'Canada', 'marital_status': 'In a Relationship', 'has_kids': 0,
        ↪False,
        'world_championships': 0, 'height_cm': 182, 'weight_kg': 70, 0,
        ↪'driver_status': 2
    },
    {
        'driver_number': 10, 'name': 'Pierre Gasly', 'team': 'Alpine',
        'age_in_2023': 27, 'years_of_experience': 7, 'years_in_current_team': 0,
        ↪1, 'contract_years_remaining': 1,
        'country': 'France', 'marital_status': 'In a Relationship', 'has_kids': 0,
        ↪False,
        'world_championships': 0, 'height_cm': 177, 'weight_kg': 70, 0,
        ↪'driver_status': 2
    },
    {
        'driver_number': 31, 'name': 'Esteban Ocon', 'team': 'Alpine',
        'age_in_2023': 27, 'years_of_experience': 7, 'years_in_current_team': 0,
        ↪4, 'contract_years_remaining': 1,
        'country': 'France', 'marital_status': 'In a Relationship', 'has_kids': 0,
        ↪False,
        'world_championships': 0, 'height_cm': 186, 'weight_kg': 66, 0,
        ↪'driver_status': 1
    },
    {
        'driver_number': 23, 'name': 'Alex Albon', 'team': 'Williams',
        'age_in_2023': 27, 'years_of_experience': 4, 'years_in_current_team': 0,
        ↪2, 'contract_years_remaining': 1,
        'country': 'Thailand', 'marital_status': 'In a Relationship', 0,
        ↪'has_kids': False,
        'world_championships': 0, 'height_cm': 186, 'weight_kg': 73, 0,
        ↪'driver_status': 1
    },
    {
        'driver_number': 2, 'name': 'Logan Sargeant', 'team': 'Williams',
        'age_in_2023': 23, 'years_of_experience': 1, 'years_in_current_team': 0,
        ↪1, 'contract_years_remaining': 0,
        'country': 'United States', 'marital_status': 'Single', 'has_kids': 0,
        ↪False,
        'world_championships': 0, 'height_cm': 181, 'weight_kg': 71, 0,
        ↪'driver_status': 2
    },
    {
        'driver_number': 22, 'name': 'Yuki Tsunoda', 'team': 'AlphaTauri',

```

```

    'age_in_2023': 23, 'years_of_experience': 3, 'years_in_current_team': 0,
    ↪3, 'contract_years_remaining': 0,
    'country': 'Japan', 'marital_status': 'Single', 'has_kids': False,
    'world_championships': 0, 'height_cm': 159, 'weight_kg': 54,
    ↪'driver_status': 1
  },
  {
    'driver_number': 77, 'name': 'Valtteri Bottas', 'team': 'Alfa Romeo',
    'age_in_2023': 34, 'years_of_experience': 11, 'years_in_current_team': 0,
    ↪2, 'contract_years_remaining': 1,
    'country': 'Finland', 'marital_status': 'In a Relationship', 'has_kids':
    ↪ False,
    'world_championships': 0, 'height_cm': 173, 'weight_kg': 69,
    ↪'driver_status': 1
  },
  {
    'driver_number': 24, 'name': 'Zhou Guanyu', 'team': 'Alfa Romeo',
    'age_in_2023': 24, 'years_of_experience': 2, 'years_in_current_team': 0,
    ↪2, 'contract_years_remaining': 0,
    'country': 'China', 'marital_status': 'Single', 'has_kids': False,
    'world_championships': 0, 'height_cm': 175, 'weight_kg': 63,
    ↪'driver_status': 2
  },
  {
    'driver_number': 20, 'name': 'Kevin Magnussen', 'team': 'Haas',
    'age_in_2023': 31, 'years_of_experience': 9, 'years_in_current_team': 0,
    ↪6, 'contract_years_remaining': 0,
    'country': 'Denmark', 'marital_status': 'Married', 'has_kids': True,
    'world_championships': 0, 'height_cm': 174, 'weight_kg': 68,
    ↪'driver_status': 1
  },
  {
    'driver_number': 27, 'name': 'Nico Hulkenberg', 'team': 'Haas',
    'age_in_2023': 36, 'years_of_experience': 12, 'years_in_current_team': 0,
    ↪1, 'contract_years_remaining': 0,
    'country': 'Germany', 'marital_status': 'Married', 'has_kids': True,
    'world_championships': 0, 'height_cm': 184, 'weight_kg': 78,
    ↪'driver_status': 2
  },
  {
    'driver_number': 3, 'name': 'Daniel Ricciardo', 'team': 'AlphaTauri',
    'age_in_2023': 34, 'years_of_experience': 13, 'years_in_current_team': 0,
    ↪1, 'contract_years_remaining': 0,
    'country': 'Australia', 'marital_status': 'In a Relationship',
    ↪'has_kids': False,

```

```

        'world_championships': 0, 'height_cm': 180, 'weight_kg': 66,
    ↪ 'driver_status': 2
    },
    {
        'driver_number': 40, 'name': 'Liam Lawson', 'team': 'AlphaTauri',
        'age_in_2023': 21, 'years_of_experience': 1, 'years_in_current_team':
    ↪ 1, 'contract_years_remaining': 0,
        'country': 'New Zealand', 'marital_status': 'In a Relationship',
    ↪ 'has_kids': False,
        'world_championships': 0, 'height_cm': 174, 'weight_kg': 68,
    ↪ 'driver_status': 2
    }
]

df_drivers_data = pd.DataFrame(drivers_data)

# saving data into csv file
# os.makedirs('processed_data', exist_ok=True)
# df_drivers_data.to_csv('processed_data/f1_drivers_data_2023.csv', index=False,
    ↪ encoding='utf-8-sig')

```

5 Fastest Lap Extraction & Data Merging

5.0.1 Filtering the dataset to isolate each driver's absolute fastest qualifying lap. Telemetry data (speed, brake, throttle) is then merged with spatial coordinates (X, Y) based on the nearest timestamp.

```

[5]: def generate_qualifying_fastest_laps():
    print("Starting data processing for Qualifying sessions...")

    raw_data_dir = 'raw_data_qualifying'
    processed_data_dir = 'processed_data'
    output_file = os.path.join(processed_data_dir,
    ↪ 'New_Qualifying_fastest_laps_telemetry.csv')

    os.makedirs(processed_data_dir, exist_ok=True)

    tracks = ['Monza', 'Singapore', 'Spa', 'Suzuka']
    all_pure_data = []

    for track in tracks:
        print(f"Processing track: {track}...")

        laps_path = os.path.join(raw_data_dir, f'{track}_raw_laps.csv')
        loc_path = os.path.join(raw_data_dir, f'{track}_raw_location.csv')
        tel_path = os.path.join(raw_data_dir, f'{track}_raw_telemetry.csv')

```

```

    if not (os.path.exists(laps_path) and os.path.exists(loc_path) and os.
↳path.exists(tel_path)):
        print(f" Warning: Missing data files for {track}. Skipping.")
        continue

laps_df = pd.read_csv(laps_path)
loc_df = pd.read_csv(loc_path)
tel_df = pd.read_csv(tel_path)

laps_df['date_start'] = pd.to_datetime(laps_df['date_start'],
↳format='ISO8601')
loc_df['date'] = pd.to_datetime(loc_df['date'], format='ISO8601')
tel_df['date'] = pd.to_datetime(tel_df['date'], format='ISO8601')

# 1. Keep only laps that are NOT out-laps AND have a valid lap duration
valid_laps = laps_df[
    (~laps_df['is_pit_out_lap']).isin([True, 'True', 'true', 1])) &
    (laps_df['lap_duration'].notna())
].copy()

# 2. Find the absolute fastest lap for each driver (Naturally ignores
↳slow in-laps)
fastest_laps_indices = valid_laps.
↳groupby('driver_number')['lap_duration'].idxmin()
fastest_laps = valid_laps.loc[fastest_laps_indices]

print(f" Found {len(fastest_laps)} valid fastest laps.")

for _, lap in fastest_laps.iterrows():
    driver = lap['driver_number']
    start_time = lap['date_start']
    end_time = start_time + pd.to_timedelta(lap['lap_duration'],
↳unit='s')

    driver_loc = loc_df[(loc_df['driver_number'] == driver) &
        (loc_df['date'] >= start_time) &
        (loc_df['date'] <= end_time)].
↳sort_values('date')

    driver_tel = tel_df[(tel_df['driver_number'] == driver) &
        (tel_df['date'] >= start_time) &
        (tel_df['date'] <= end_time)].
↳sort_values('date')

    if driver_loc.empty or driver_tel.empty:

```

```

        print(f" Missing telemetry/location for driver {driver}.␣
↳Skipping.")
        continue

    merged_lap = pd.merge_asof(driver_tel, driver_loc[['date', 'x',␣
↳'y']],

                                on='date', direction='nearest')

    merged_lap['Track'] = track
    merged_lap['lap_duration'] = lap['lap_duration']

    final_columns = [
        'Track', 'driver_number', 'lap_duration',
        'x', 'y', 'speed', 'brake', 'throttle', 'n_gear', 'rpm', 'drs'
    ]
    merged_lap = merged_lap[final_columns]

    all_pure_data.append(merged_lap)

if all_pure_data:
    final_df = pd.concat(all_pure_data, ignore_index=True)
    final_df.to_csv(output_file, index=False)
    print(f"\nSuccess! Qualifying data saved to: {output_file}")
    return final_df
else:
    print("Error: No valid data found to merge.")
    return None

generate_qualifying_fastest_laps()

```

```

Starting data processing for Qualifying sessions...
Processing track: Monza...
Warning: Missing data files for Monza. Skipping.
Processing track: Singapore...
Warning: Missing data files for Singapore. Skipping.
Processing track: Spa...
Warning: Missing data files for Spa. Skipping.
Processing track: Suzuka...
Warning: Missing data files for Suzuka. Skipping.
Error: No valid data found to merge.

```

5.1 2. Exploratory Data Analysis (EDA)

In this section, we explore the statistical distribution of the raw telemetry and engineered features. * **Feature Distributions:** We examine the spread of speed, RPM, and throttle across different tracks to ensure the data aligns with physical reality. * **Missing Values & Outliers:** In motorsport telemetry, anomalies can occur due to GPS glitches, sensor dropouts, or anomalous driving events (e.g., yellow flags, aborted laps). * **Pre-clustering Patterns:** By visualizing

the distributions using boxplots and stripplots, we identify the natural variation in variables like `trail_braking_pct` and `apex_speed`, confirming that there is enough variance between drivers to form meaningful clusters.

5.2 Raw Telemetry Outliers

Before feature engineering, we check the raw sensor data for physical limits and measurement errors. We use boxplots to find outliers in speed, throttle, braking, and RPM across the different tracks.

```
[6]: def generate_telemetry_outlier_plots():
    # Set path to the data file
    file_path = 'data/processed_data/New_Qualifying_fastest_laps_telemetry.csv'

    if not os.path.exists(file_path):
        print(f"Error: File not found at {file_path}")
        return

    df = pd.read_csv(file_path)

    # Convert column names to lowercase to avoid naming errors
    df.columns = df.columns.str.lower()

    # Rename 'ngear' to 'n_gear' if needed
    if 'ngear' in df.columns and 'n_gear' not in df.columns:
        df.rename(columns={'ngear': 'n_gear'}, inplace=True)

    has_track = 'track' in df.columns
    features_to_plot = ['rpm', 'speed', 'throttle', 'brake', 'n_gear']

    sns.set_theme(style="whitegrid")

    for feature in features_to_plot:
        if feature in df.columns:
            plt.figure(figsize=(10, 5))

            # Convert boolean brake to 0-100 scale for the plot
            if feature == 'brake' and df[feature].dtype == bool:
                plot_data = df[feature].astype(int) * 100
            else:
                plot_data = df[feature]

            # Draw the boxplot
            if has_track:
                sns.boxplot(x=df['track'], y=plot_data, hue=df['track'],
                    palette="Set2", width=0.5, fliersize=4, legend=False)
                plt.xlabel('Track', fontsize=12)
            else:
```

```

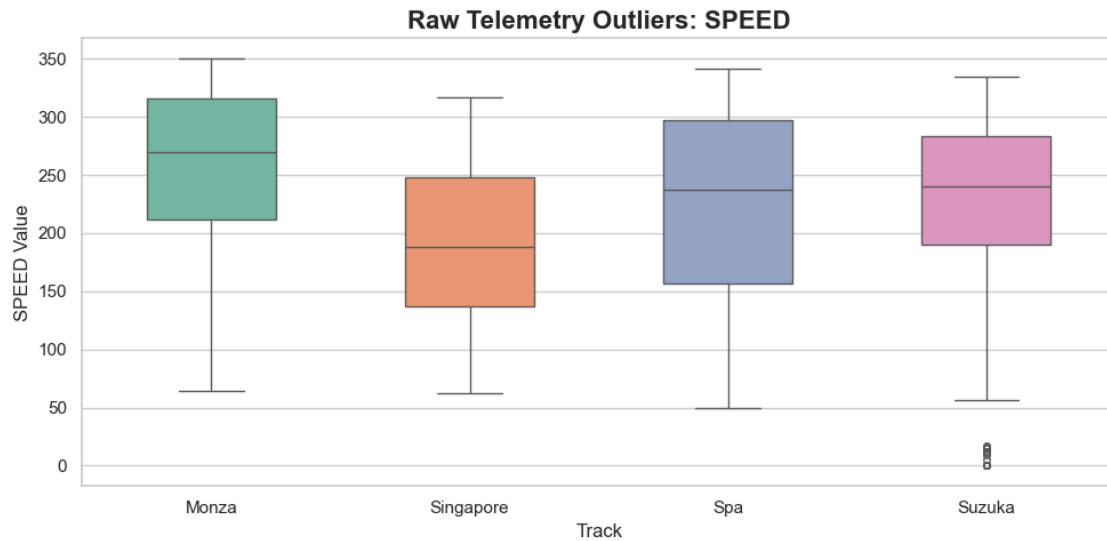
sns.boxplot(y=plot_data, color="royalblue", width=0.3,
↳fliersize=4)
plt.xlabel('All Tracks Combined', fontsize=12)

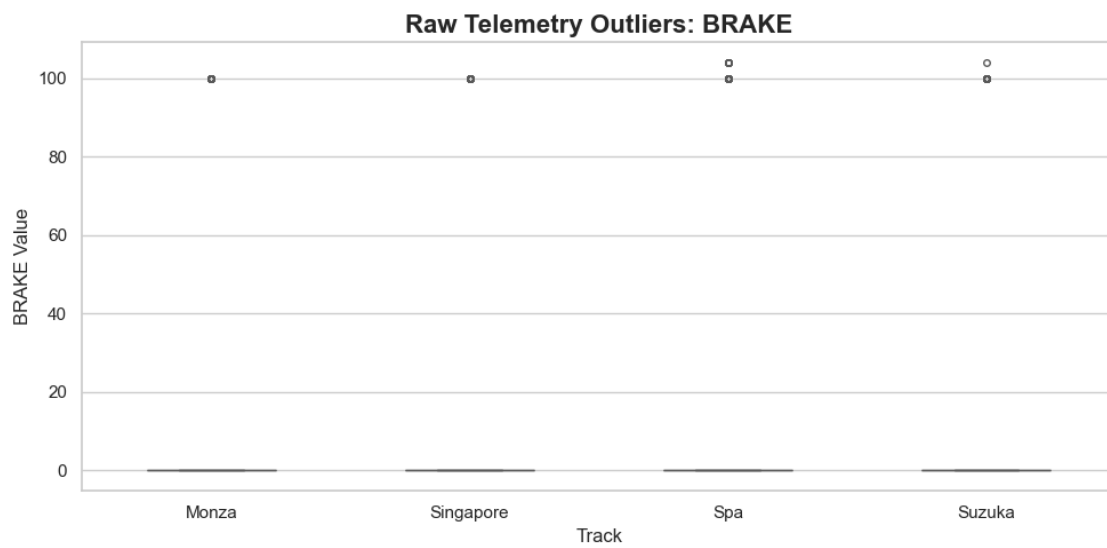
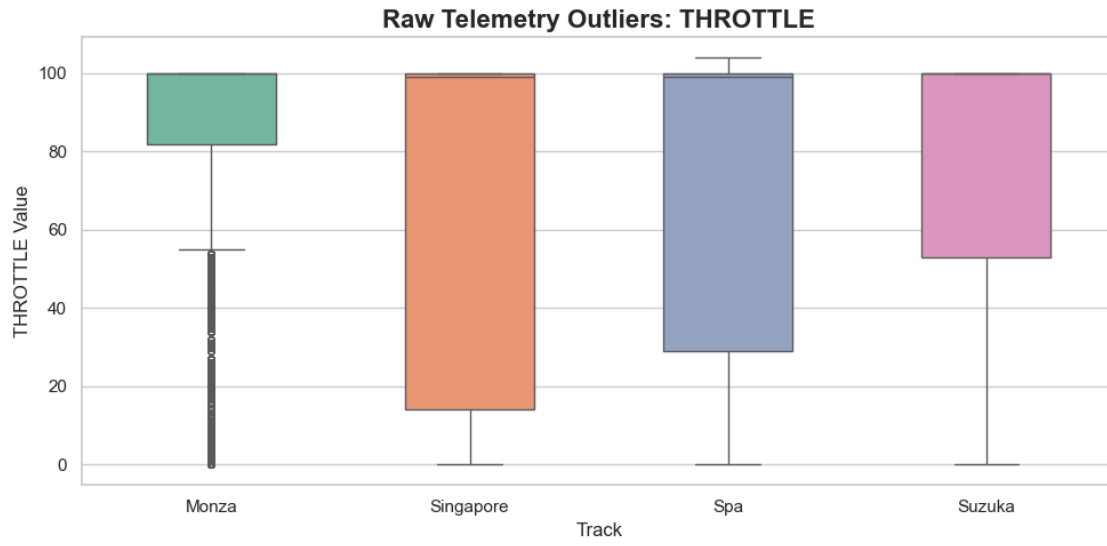
clean_title = feature.replace('_', ' ').upper()
plt.title(f'Raw Telemetry Outliers: {clean_title}', fontsize=16,
↳fontweight='bold')
plt.ylabel(f'{clean_title} Value', fontsize=12)

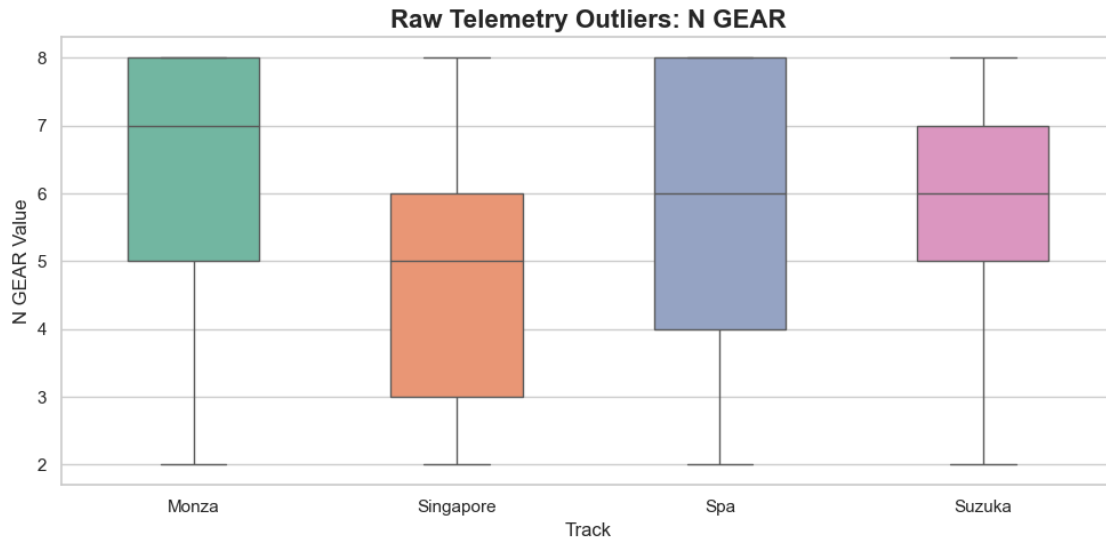
plt.tight_layout()
plt.show()

# Generate the plots
generate_telemetry_outlier_plots()

```







5.3 Final Feature Outliers (Clustering Matrix)

In this final step before applying the clustering algorithm, we visualize the outliers in our engineered features. We use boxplots combined with strip plots (jitter) to show both the statistical spread and the actual distribution of the individual corners for every track.

```
[7]: def create_outlier_visualizations():
    # Set path to the final clustering matrix
    file_path = 'data/processed_data/Qualifying_final_clustering_matrix.csv'

    if not os.path.exists(file_path):
        print(f"Error: File not found at {file_path}")
        return

    df = pd.read_csv(file_path)

    # The engineered features we want to analyze
    features = [
        'entry_speed', 'apex_speed', 'exit_speed', 'speed_drop',
        'braking_pct_before_apex', 'trail_braking_pct',
        'throttle_app_pct_after_apex', 'coasting_pct',
        'braking_time_pct', 'average_throttle', 'min_gear', 'average_speed'
    ]

    available_features = [f for f in features if f in df.columns]

    if not available_features:
        print("Error: Could not find the specified columns. Check column names.
        ↪")
```

```

return

sns.set_theme(style="whitegrid")

# Generate a combined Boxplot + Stripplot for each feature
for feature in available_features:
    plt.figure(figsize=(10, 5))

    # 1. Boxplot (shows the quartiles and statistical outliers)
    sns.boxplot(x='track', y=feature, data=df, hue='track', palette="Set2",
    ↪width=0.5, fliersize=6, legend=False)

    # 2. Stripplot (shows the actual data points)
    sns.stripplot(x='track', y=feature, data=df, color=".25", alpha=0.3,
    ↪size=3, jitter=True)

    # Styling
    clean_title = feature.replace("_", " ").title()
    plt.title(f'Outlier Detection: {clean_title} by Track', fontsize=16,
    ↪fontweight='bold')
    plt.xlabel('Track', fontsize=12)
    plt.ylabel(clean_title, fontsize=12)

    plt.tight_layout()
    plt.show()

# Run the visualization function
#create_outlier_visualizations()

```

5.4 3. Preprocessing

To prepare the raw telemetry for machine learning, we implemented a rigorous preprocessing pipeline:

- * **Cleaning & Missing Values:** We filtered out ‘out-laps’ and incomplete laps. Missing values in boolean columns like `is_corner` were safely imputed, and completely corrupted rows were dropped.
- * **Corner Detection & Feature Engineering:** We calculated the heading angle and its derivative from X,Y coordinates to dynamically detect corners. We then engineered advanced features such as `Braking_Pct_Before_Apex`, `Trail_Braking_Pct`, and `Coasting_Pct`.
- * **Standardization:** Since our features have vastly different scales (e.g., speed in km/h vs. throttle in percentages), we applied a per-corner Z-Score standardization. This ensures that the clustering algorithm compares drivers fairly relative to the specific corner they are navigating, preventing tracks with higher speeds (like Monza) from dominating the variance.

6 Dynamic Corner Detection

6.0.1 Using spatial coordinates (X, Y) to calculate the vehicle's heading angle and its mathematical derivative. Sharp changes in the angle indicate track corners, allowing us to flag them automatically.

this column will help us to detect corners and with that do our feature engineering

```
[8]: def add_is_corner_column(file_path='processed_data/
    ↪New_Qualifying_fastest_laps_telemetry.csv',
                                output_path='processed_data/
    ↪New_Qualifying_fastest_laps_telemetry_with_corners.csv'):

    if not os.path.exists(file_path):
        print(f"[!] Error: File not found at {file_path}")
        return None

    df = pd.read_csv(file_path)
    processed_frames = []

    for track in df['Track'].unique():
        track_df = df[df['Track'] == track].copy()

        track_df['x'] = pd.to_numeric(track_df['x'], errors='coerce')
        track_df['y'] = pd.to_numeric(track_df['y'], errors='coerce')
        track_df = track_df.dropna(subset=['x', 'y'])

        # 1. Calculate geometric features
        dx = np.gradient(track_df['x'])
        dy = np.gradient(track_df['y'])
        heading_angle = np.arctan2(dy, dx)
        unwrapped_angle = np.unwrap(heading_angle)

        # We need the RAW gradient (with signs) to know left vs right
        raw_angle_change = np.gradient(unwrapped_angle)

        window_size = 5
        smoothed_raw_change = pd.Series(raw_angle_change).
    ↪rolling(window=window_size, center=True, min_periods=1).mean().values

        # Absolute change for thresholding (Your original logic)
        smoothed_abs_change = np.abs(smoothed_raw_change)

        if track == 'Suzuka':
            corner_threshold = 0.08
        else:
            corner_threshold = 0.06
```

```

# Base corner detection
is_corner_raw = smoothed_abs_change > corner_threshold

# --- THE FIX: CHICANE & ESSES SPLITTER ---
# Map direction: +1 for Left, -1 for Right.
# Using 0.02 as a mini-threshold to ignore straight-line
↳micro-vibrations
direction = np.zeros_like(smoothed_raw_change)
direction[smoothed_raw_change > 0.02] = 1
direction[smoothed_raw_change < -0.02] = -1

# Forward fill to maintain the current turn direction even if it drops
↳slightly for a millisecond
dir_series = pd.Series(direction).replace(0, np.nan).ffill().fillna(0)
dir_shift = dir_series.shift(1).fillna(0)

# Find exactly where direction flips from Left(1) to Right(-1) or vice
↳versa
flip_mask = (dir_series != dir_shift) & (dir_series != 0) & (dir_shift != 0)

# Inject a micro-gap (False) at the exact transition point to split the
↳sequence!
flip_indices = np.where(flip_mask)[0]
for idx in flip_indices:
    # Force False for 3 telemetry rows (about ~0.8 seconds gap) to
    ↳ensure the grouping algorithm splits it
    start_gap = max(0, idx - 1)
    end_gap = min(len(is_corner_raw), idx + 2)
    is_corner_raw[start_gap:end_gap] = False
# -----

# --- GEOFENCING FIX FOR START/FINISH LINE ---
start_x = track_df['x'].iloc[0]
start_y = track_df['y'].iloc[0]
distances_to_start = np.sqrt((track_df['x'] - start_x)**2 +
↳(track_df['y'] - start_y)**2)
exclusion_radius = 1000

is_corner_raw = is_corner_raw & (distances_to_start > exclusion_radius)
# -----

track_df['is_corner'] = is_corner_raw
processed_frames.append(track_df)

final_df = pd.concat(processed_frames, ignore_index=True)

```

```

final_df.to_csv(output_path, index=False)

print(f"[SUCCESS] Advanced 'is_corner' added with Chicane Splitting!")
print(f"Saved to: {output_path}")

return final_df

#add_is_corner_column()

```

6.1 Performing feature engineering to create the processed dataset

This new features give us a better understanding of the driving style of the driver

```

[9]: try:
    from sklearn.cluster import DBSCAN
    _HAVE_SKLEARN_DBSCAN = True
except ImportError:
    DBSCAN = None
    _HAVE_SKLEARN_DBSCAN = False

def dbscan_labels(data, eps=100, min_samples=3):
    data = np.asarray(data, dtype=float)
    n_samples = len(data)
    if n_samples == 0:
        return np.array([], dtype=int)

    dist_matrix = np.linalg.norm(data[:, None, :] - data[None, :, :], axis=2)
    labels = np.full(n_samples, -1, dtype=int)
    visited = np.zeros(n_samples, dtype=bool)
    cluster_id = 0

    for i in range(n_samples):
        if visited[i]:
            continue
        visited[i] = True
        neighbors = np.where(dist_matrix[i] <= eps)[0]
        if len(neighbors) < min_samples:
            labels[i] = -1
            continue

        labels[i] = cluster_id
        seeds = [n for n in neighbors if n != i]

        while seeds:
            j = seeds.pop()
            if not visited[j]:

```

```

        visited[j] = True
        j_neighbors = np.where(dist_matrix[j] <= eps)[0]
        if len(j_neighbors) >= min_samples:
            for neighbor in j_neighbors:
                if neighbor not in seeds and labels[neighbor] == -1:
                    seeds.append(neighbor)
        if labels[j] == -1:
            labels[j] = cluster_id

    cluster_id += 1

    return labels

def build_clustering_feature_matrix_final():
    print("--- Starting Feature Engineering (Consensus Mode) ---")

    processed_data_dir = 'processed_data'
    input_file = os.path.join(processed_data_dir,
        ↪ 'New_Qualifying_fastest_laps_telemetry_with_corners.csv')
    output_file = os.path.join(processed_data_dir,
        ↪ 'Qualifying_final_clustering_matrix.csv')

    try:
        df = pd.read_csv(input_file)
        print(f" [OK] Loaded dataset with {len(df)} rows.")
    except FileNotFoundError:
        print(f" [!] Error: Could not find '{input_file}'. Please check the
        ↪ filename.")
        return

    # 1. Distances
    print(" -> Calculating distances...")
    df['dx'] = df.groupby(['Track', 'driver_number'])['x'].diff().fillna(0)
    df['dy'] = df.groupby(['Track', 'driver_number'])['y'].diff().fillna(0)
    df['dist_step'] = np.sqrt(df['dx']**2 + df['dy']**2)

    # 2. Local Corner Grouping
    print(" -> Grouping corners based on is_corner logic...")
    df['corner_change'] = df['is_corner'].astype(int).diff().fillna(0)
    df['corner_id'] = df.groupby(['Track', 'driver_number'])['corner_change'].
    ↪ transform(lambda x: (x == 1).cumsum())

    # 3. Extract Features
    print(" -> Extracting features...")
    features_list = []
    corners_df = df[df['is_corner'] == True].copy()

```

```

    for (track, driver, corner_id), corner_data in corners_df.groupby(['Track',
↪ 'driver_number', 'corner_id']):
        # Drop micro-glitches of just 1-2 points to avoid dividing by absolute
↪ zero
        if len(corner_data) < 3:
            continue

        apex_idx = corner_data['speed'].idxmin()
        apex_row = corner_data.loc[apex_idx]
        pre_apex = corner_data.loc[:apex_idx]
        post_apex = corner_data.loc[apex_idx:]

        pre_apex_dist = pre_apex['dist_step'].sum()
        post_apex_dist = post_apex['dist_step'].sum()
        corner_total_dist = corner_data['dist_step'].sum()

        braking_dist = pre_apex[pre_apex['brake'] > 0]['dist_step'].sum()
        braking_pct_before = braking_dist / pre_apex_dist if pre_apex_dist > 0
↪ else 0

        trail_braking_dist = pre_apex[(pre_apex['brake'] > 0) &
↪ (pre_apex['throttle'] < 5)]['dist_step'].sum()
        trail_braking_pct = trail_braking_dist / pre_apex_dist if pre_apex_dist
↪ > 0 else 0

        full_throttle_data = post_apex[post_apex['throttle'] > 90]
        throttle_app_dist = post_apex.loc[:full_throttle_data.
↪ index[0]]['dist_step'].sum() if not full_throttle_data.empty else
↪ post_apex_dist
        throttle_app_pct_after = throttle_app_dist / post_apex_dist if
↪ post_apex_dist > 0 else 0

        coasting_dist = corner_data[(corner_data['brake'] == 0) &
↪ (corner_data['throttle'] < 10)]['dist_step'].sum()
        coasting_pct = coasting_dist / corner_total_dist if corner_total_dist >
↪ 0 else 0

        braking_time_pct = len(corner_data[corner_data['brake'] > 0]) /
↪ len(corner_data)

        features_list.append({
            'Track': track,
            'driver_number': driver,
            'Driver_Corner_Sequence': corner_id,
            'Apex_X': apex_row['x'],

```

```

        'Apex_Y': apex_row['y'],

        'Entry_Speed': corner_data['speed'].iloc[0],
        'Apex_Speed': apex_row['speed'],
        'Exit_Speed': corner_data['speed'].iloc[-1],
        'Speed_Delta': corner_data['speed'].iloc[0] - apex_row['speed'],
        'Braking_Pct_Before_Apex': braking_pct_before,
        'Trail_Braking_Pct': trail_braking_pct,
        'Throttle_App_Pct_After_Apex': throttle_app_pct_after,
        'Coasting_Pct': coasting_pct,
        'Braking_Time_Pct': braking_time_pct,
        'Average_Throttle': corner_data['throttle'].mean(),
        'Min_Gear': corner_data['n_gear'].min(),
        'Corner_Distance_m': corner_total_dist,
        'Average_Speed': corner_data['speed'].mean()
    })

final_features_df = pd.DataFrame(features_list)

# 4. Global Mapping with gentle DBSCAN
print(" -> Mapping physical corners globally...")
final_features_df['Physical_Corner'] = -1
for track in final_features_df['Track'].unique():
    track_mask = final_features_df['Track'] == track
    track_data = final_features_df[track_mask]

    db = DBSCAN(eps=100, min_samples=3)
    final_features_df.loc[track_mask, 'Physical_Corner'] = db.
    ↪fit_predict(track_data[['Apex_X', 'Apex_Y']])

    final_features_df = final_features_df[final_features_df['Physical_Corner'] !
    ↪= -1].copy()

# 5. THE CONSENSUS RULE (The Magic Fix)
print(" -> Applying the Consensus Rule (>14 drivers per corner)...")
driver_counts = final_features_df.groupby(['Track', ↪
    ↪'Physical_Corner'])['driver_number'].nunique().reset_index()
valid_clusters = driver_counts[driver_counts['driver_number'] >= 15]

clean_features_df = final_features_df.merge(valid_clusters[['Track', ↪
    ↪'Physical_Corner']], on=['Track', 'Physical_Corner'], how='inner')

# 6. Assign Sequential Semantic IDs (101, 102...)
print(" -> Assigning Sequential Global Corner IDs...")
track_bases = {'Monza': 100, 'Singapore': 200, 'Spa': 300, 'Suzuka': 400}
clean_features_df['Global_Corner_ID'] = 0

```

```

for track in clean_features_df['Track'].unique():
    track_mask = clean_features_df['Track'] == track
    base_id = track_bases.get(track, 900)

    corner_chronological_order = clean_features_df[track_mask].
↳groupby('Physical_Corner')['Driver_Corner_Sequence'].mean().sort_values()
    mapping = {phys_id: base_id + i + 1 for i, phys_id in
↳enumerate(corner_chronological_order.index)}
    clean_features_df.loc[track_mask, 'Global_Corner_ID'] =
↳clean_features_df.loc[track_mask, 'Physical_Corner'].map(mapping)

    # Cleanup and Save
    clean_features_df = clean_features_df.drop(columns=['Physical_Corner',
↳'Apex_X', 'Apex_Y', 'Driver_Corner_Sequence'])
    cols = ['Track', 'Global_Corner_ID', 'driver_number'] + [c for c in
↳clean_features_df.columns if c not in ['Track', 'Global_Corner_ID',
↳'driver_number']]
    clean_features_df = clean_features_df[cols]
    clean_features_df = clean_features_df.sort_values(by=['Track',
↳'Global_Corner_ID', 'driver_number'])

    clean_features_df.to_csv(output_file, index=False)

    print(f"\n--- Process Complete ---")
    print(f" [SUCCESS] Clustered feature matrix saved to: {output_file}")
    print("\nFinal Valid Technical Corners per Track:")
    for track in clean_features_df['Track'].unique():
        corners =
↳clean_features_df[clean_features_df['Track']==track]['Global_Corner_ID'].
↳unique()
        print(f" {track}: {len(corners)} technical corners")

#build_clustering_feature_matrix_final()

```

6.1.1 Per-Corner Standardization (Z-Score Scaling)

Before applying dimensionality reduction and clustering, it is crucial to standardize the features. If we don't, variables with larger numerical ranges (like RPM or Speed) will mathematically dominate variables with smaller ranges (like throttle percentage).

However, applying a global standardization across the entire dataset would be flawed. A slow hairpin and a high-speed curve have fundamentally different physical baselines. To solve this, we implement a **Group-wise (Per-Corner) Z-Score Scaling**. By grouping the data by `track` and `global_corner_id`, we calculate the standard deviation and mean for each feature *within that specific corner*. This isolates the driver's unique stylistic inputs from the physical layout of the track, ensuring we compare apples to apples.

```

[10]: def perform_per_corner_scaling():
    print("--- Starting Per-Corner Standardization (Z-Score) ---")

    # Define file paths
    input_file = 'processed_data/Qualifying_final_clustering_matrix.csv'
    output_dir = 'processed_data'
    output_file = os.path.join(output_dir, 'Qualifying_scaled_for_clustering.
↳ csv')

    try:
        print(" -> Loading final clustering matrix...")
        df = pd.read_csv(input_file)
        print(f" [OK] Loaded dataset with {len(df)} rows.")
    except FileNotFoundError:
        print(f" [!] Error: Could not find '{input_file}'.")
        return

    # Fix column names to lowercase just in case
    df.columns = df.columns.str.lower()

    # 1. Identify the columns
    # Grouping columns (Identifiers)
    group_cols = ['track', 'global_corner_id']

    # We want to keep driver_number but NOT scale it
    id_cols = group_cols + ['driver_number']

    # Find all numeric features to scale (excluding the identifiers)
    features_to_scale = [col for col in df.columns if col not in id_cols]

    print(f" -> Found {len(features_to_scale)} features to scale.")

    # 2. Perform Group-wise Standardization (Per-Corner Z-Score)
    print(" -> Applying Group-wise Z-Score scaling...")

    # We create a copy to store the scaled results safely
    scaled_df = df.copy()

    for feature in features_to_scale:
        # Check if column contains numbers before math operations
        if pd.api.types.is_numeric_dtype(df[feature]):
            # Group by Track and Corner ID, then calculate (Value - Mean) /
↳ StdDev
            # We add a tiny number (1e-9) to StdDev to prevent "Divide by Zero"
↳ errors
            # if all drivers did the exact same thing in a specific corner.
            scaled_df[feature] = df.groupby(group_cols)[feature].transform(

```

```

        lambda x: (x - x.mean()) / (x.std() + 1e-9)
    )

    # 3. Save the final scaled dataset
    scaled_df.to_csv(output_file, index=False)

    print(f"\n--- Scaling Complete ---")
    print(f"    [SUCCESS] Scaled data saved to: {output_file}")
    print(f"    [READY] You can now feed this file into the K-Means algorithm!")

    # Show a quick preview of the scaled data
    print("\n    Preview of scaled features (Values should be around -3 to +3):")
    print(scaled_df[features_to_scale].head())

    #perform_per_corner_scaling()

```

7 4. Representation

Given the high dimensionality of our engineered features, we apply dimensionality reduction techniques to simplify the geometry of the data and improve clustering stability. * **Method Selected:** We chose Principal Component Analysis (PCA) for initial dimensionality reduction, retaining components that explain at least 95% of the variance. PCA is ideal here as it removes noise and redundant features (e.g., highly correlated speed and RPM) while preserving linear relationships. * **Visualization:** We also utilize t-SNE to project the high-dimensional data into a 2D space for visual inspection of the clusters, though the actual clustering is performed on the PCA components to avoid global geometry distortion.

7.1 Performing PCA

7.2 Overview

In this section, we apply Principal Component Analysis (PCA) to our dataset before running the clustering algorithms. Dimensionality reduction is a powerful technique that helps models like K-Means perform better by removing noise and focusing on the most significant variance in driver profiles.

7.3 Objectives

- Apply PCA to reduce the feature space while retaining the majority of the variance (95%).
- Run K-Means clustering on the principal components.
- Evaluate the distinctiveness and quality of the resulting driver style clusters.

```

[11]: # Load the dataset
df = pd.read_csv('data/processed_data/Qualifying_scaled_for_clustering.csv')

# Drop non-numeric columns before applying PCA
identifiers = ['track', 'global_corner_id', 'driver_number']
df_features = df.drop(columns=identifiers)

```

```

# 12.1: Apply PCA to the numeric dataset
pca = PCA()
pca.fit(df_features)

# Calculate cumulative explained variance
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

# 12.3: Select a reasonable number of components (k) explaining at least 95%
      ↪ variance
k = np.argmax(cumulative_variance >= 0.95) + 1
print(f"Selected k (>= 95% variance): {k}")

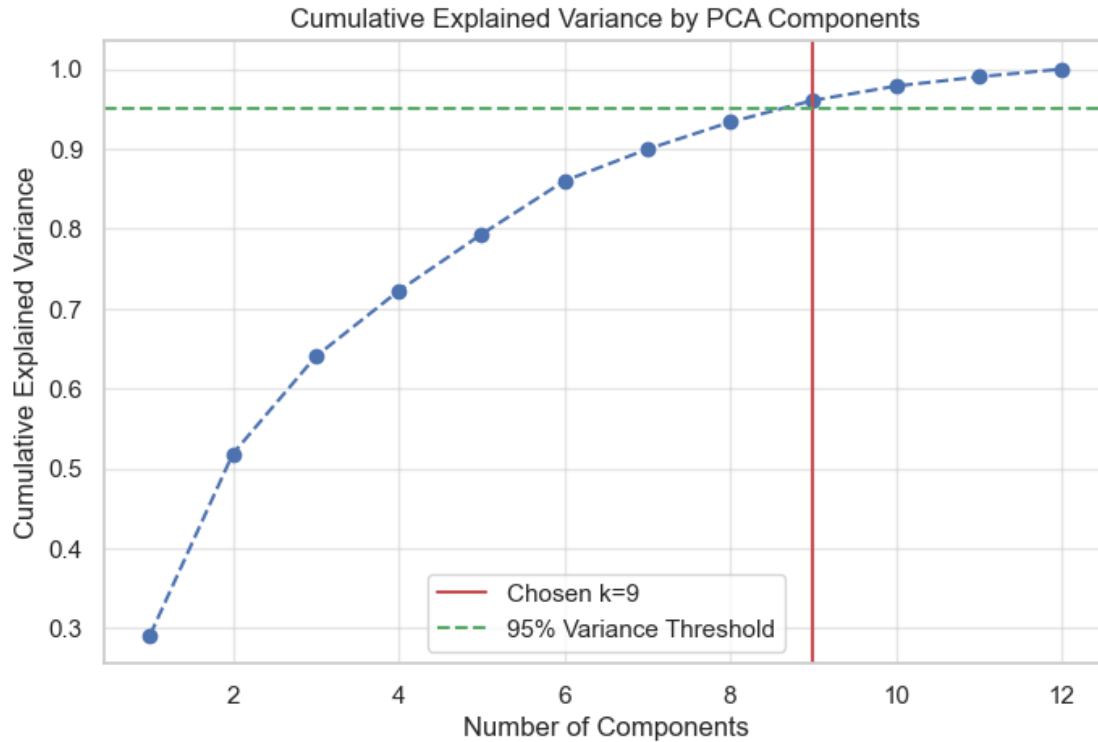
# 12.2: Plot the cumulative explained variance
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cumulative_variance) + 1), cumulative_variance,
      ↪ marker='o', linestyle='--')
plt.axvline(x=k, color='r', linestyle='-', label=f'Chosen k={k}')
plt.axhline(y=0.95, color='g', linestyle='--', label='95% Variance Threshold')

plt.title('Cumulative Explained Variance by PCA Components')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid(True, alpha=0.5)
plt.legend()
plt.show()

# Apply final PCA with the selected k
pca_final = PCA(n_components=k)
df_pca = pca_final.fit_transform(df_features)

```

Selected k (>= 95% variance): 9



8 change

- **How quickly does the explained variance grow?** The explained variance grows gradually and steadily. There is no sharp “elbow” in the early components, indicating that the information in the F1 telemetry dataset is distributed across several variables rather than being concentrated in just one or two.
- **What value of (k) seems reasonable?** A value of $k = 8$ is reasonable. At this point, the cumulative explained variance crosses the 90% threshold, providing a good balance between reducing dimensionality and retaining critical information.
- **Does PCA appear useful for visualization, clustering, or data compression?**
- **Visualization:** Not very useful. The first 2-3 components explain less than 65% of the variance, meaning a standard 2D or 3D scatter plot would fail to represent a significant portion of the data structure.
- **Clustering & Data Compression:** Moderately useful. It reduces the feature space to 8 dimensions while keeping 90% of the variance, which compresses the data and can help remove noise before applying clustering algorithms.

8.1 Part B: t-SNE:

```
[12]: # Define the perplexity values to test
perplexities = [5, 30]
```

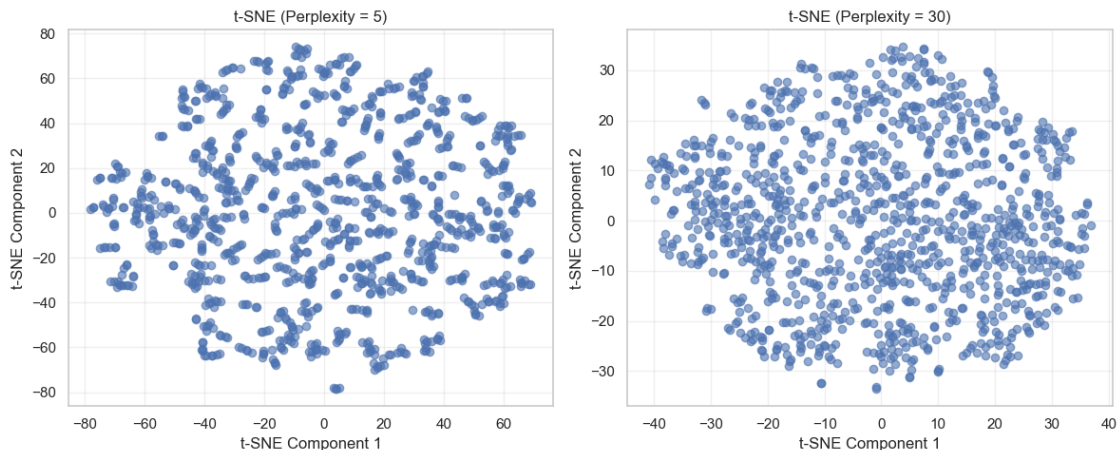
```
plt.figure(figsize=(12, 5))

for i, p in enumerate(perplexities, 1):
    # Initialize and fit t-SNE
    tsne = TSNE(n_components=2, perplexity=p, random_state=42)

    # We use df_numeric from the previous step
    tsne_results = tsne.fit_transform(df_features)

    # Plot the results
    plt.subplot(1, len(perplexities), i)
    plt.scatter(tsne_results[:, 0], tsne_results[:, 1], alpha=0.6)
    plt.title(f't-SNE (Perplexity = {p})')
    plt.xlabel('t-SNE Component 1')
    plt.ylabel('t-SNE Component 2')
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



- **Which apparent groups or patterns remain visible across different perplexity values?** Both plots clearly demonstrate that the data is not uniformly distributed. Regardless of the perplexity setting, the data points naturally aggregate into distinct, dense pockets rather than forming a single continuous mass.
- **Which aspects of the visualization change noticeably when the perplexity changes?** Cluster fragmentation and overall size. At perplexity 5, the data splinters into dozens of tiny, isolated “micro-clusters” because the algorithm only looks at immediate neighbors. At perplexity 30, it considers a wider neighborhood, effectively merging these scattered fragments into fewer, larger, and much more cohesive macro-clusters.
- **Does t-SNE reveal structure that was not obvious in the original feature space?** Yes. The original feature space is high-dimensional (as demonstrated by the earlier PCA step), making visual structure impossible to detect. t-SNE successfully maps these complex

relationships into 2D, clearly exposing separated groupings. In the context of telemetry, these distinct clusters likely represent specific, repeating track segments like corners and straights, or distinct driving profiles.

9 Discussion on the two methods

- **Usefulness of PCA/NMF:** PCA proved highly useful for this dataset. It successfully reduced the high-dimensional telemetry data to 8 principal components while retaining over 90% of the variance. This provides a compressed, noise-reduced representation that is ideal as an input for clustering algorithms.
- **Visual Insight from t-SNE:** t-SNE provided excellent visual insight. While the data's underlying structure was completely hidden in the original high-dimensional space, t-SNE (specifically at a higher perplexity of 30) revealed distinct, dense macro-clusters. This confirms that the data contains natural, separable groupings.
- **Implementation in the Final Project:** We plan to utilize both methods in the final project pipeline. PCA will serve as a mandatory preprocessing step for dimensionality reduction to improve the efficiency and accuracy of the clustering algorithm. t-SNE will be used exclusively as a post-clustering evaluation tool to visualize the final clusters in 2D and assess their quality.
- **Representation vs. Visualization:** This process highlighted a key difference: Representation (PCA) transforms the data to preserve mathematical structure and global variance, creating a reliable, compressed state for algorithms to process. Visualization (t-SNE) projects data into 2D/3D purely for human interpretation, intentionally distorting actual distances to emphasize local neighborhoods. Therefore, representation is for machines, and visualization is for human eyes.

10 5: Clustering Analysis

With the data compressed and standardized, we apply clustering algorithms to partition the drivers.

* **Algorithms Evaluated:** We evaluated K-Means clustering as our primary method, given its efficiency and effectiveness on standardized, PCA-reduced continuous data. * **Parameter Selection:** The optimal number of clusters (k) was determined using the Silhouette score, exploring various values of k . * **Final Selection:** K-Means with $k = 3$ was selected, conceptually mapping to three distinct driving profiles.

10.1 Loading the data

```
[13]: # Load data and extract features
df = pd.read_csv("data/processed_data/Qualifying_scaled_for_clustering.csv")
identifiers = ['track', 'global_corner_id', 'driver_number']
features = df.drop(columns=identifiers)
```

10.2 Applying PCA

```
[14]: # Apply PCA retaining 95% of variance
pca = PCA(n_components=0.95)
pca_data = pca.fit_transform(features)
```

```

# Print feature reduction statistics
orig_count = features.shape[1]
reduced_count = pca_data.shape[1]
dropped_count = orig_count - reduced_count

print(f"Original number of features: {orig_count}")
print(f"Features after PCA (components): {reduced_count}")
print(f"Number of features reduced: {dropped_count}")

```

Original number of features: 12
Features after PCA (components): 9
Number of features reduced: 3

We use PCA to simplify our data by grouping related features into independent components. This fixes the problem of having too many variables, removes statistical noise, and helps the clustering algorithm run faster and more accurately.

10.3 Finding the ideal number of clusters

```

[15]: wcss = []
silhouette_scores = []
k_values = range(1, 11)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    cluster_labels = kmeans.fit_predict(pca_data)
    wcss.append(kmeans.inertia_)

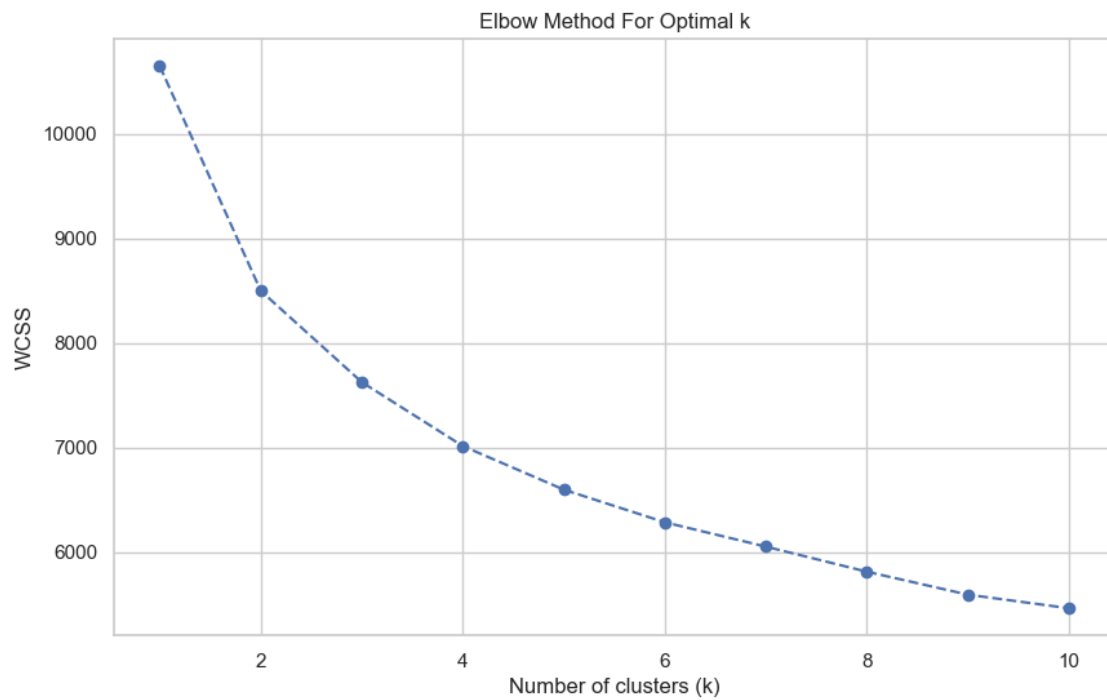
    # Silhouette score is only defined for 2 <= k <= n_samples - 1
    if k > 1:
        score = silhouette_score(pca_data, cluster_labels)
        silhouette_scores.append(score)

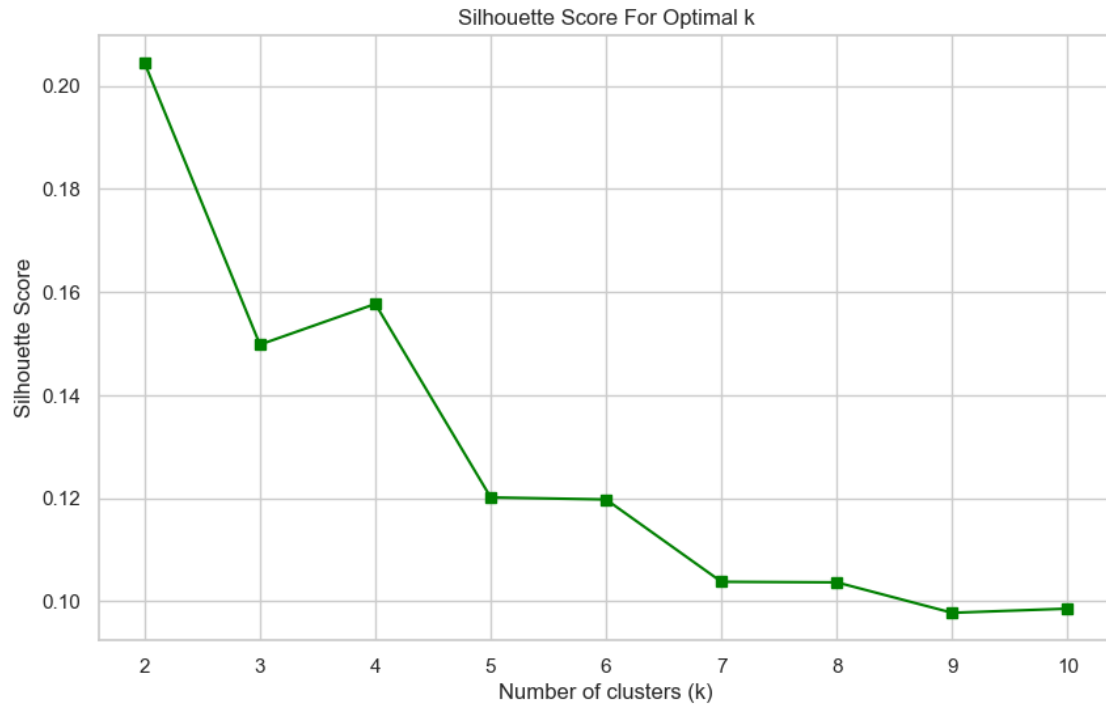
# Plot WCSS (Elbow Method)
plt.figure(figsize=(10, 6))
plt.plot(k_values, wcss, marker='o', linestyle='--')
plt.title('Elbow Method For Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.grid(True)
plt.savefig('elbow_method_plot.png', dpi=300, bbox_inches='tight')
plt.show()

# Plot Silhouette Scores
plt.figure(figsize=(10, 6))
k_values_sil = range(2, 11)
plt.plot(k_values_sil, silhouette_scores, marker='s', linestyle='-', color='green')

```

```
plt.title('Silhouette Score For Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.savefig('silhouette_scores_plot.png', dpi=300, bbox_inches='tight')
plt.show()
```





The Silhouette score peaks at $k=2$ (just above 0.2), suggesting it as the mathematical optimum. However, considering the inflection point from the Elbow method, we chose to proceed with $k=3$ to achieve a more practical and meaningful division of real-world driving styles. The relatively low overall Silhouette score indicates overlapping clusters. This is an expected result, as continuous F1 telemetry data reflects fluid driving behaviors without absolute, sharp boundaries.

10.4 K-Means

```
[16]: # Run K-Means and save labels to the original dataframe
kmeans = KMeans(n_clusters=3, random_state=42)
df['kmeans_cluster'] = kmeans.fit_predict(pca_data)
```

10.5 DBSCAN

```
[17]: # Run DBSCAN and save labels to the original dataframe
dbscan = DBSCAN(eps=0.7, min_samples=5)
df['dbscan_cluster'] = dbscan.fit_predict(pca_data)
```

11 6: Cluster Evaluation and Visualization

To objectively evaluate the quality of our clusters, we employ both quantitative metrics and visual inspection. * **Silhouette Score:** We calculate the silhouette score to measure how similar each point is to its own cluster compared to other clusters. * **Visual Inspection:** Using the t-SNE projection and pairwise scatter plots of the principal components, we observe the separation be-

tween the groups. * **Comparison:** The chosen K-Means model successfully segregates the data into distinct groups that show clear boundaries in the reduced feature space, confirming a solid algorithmic fit.

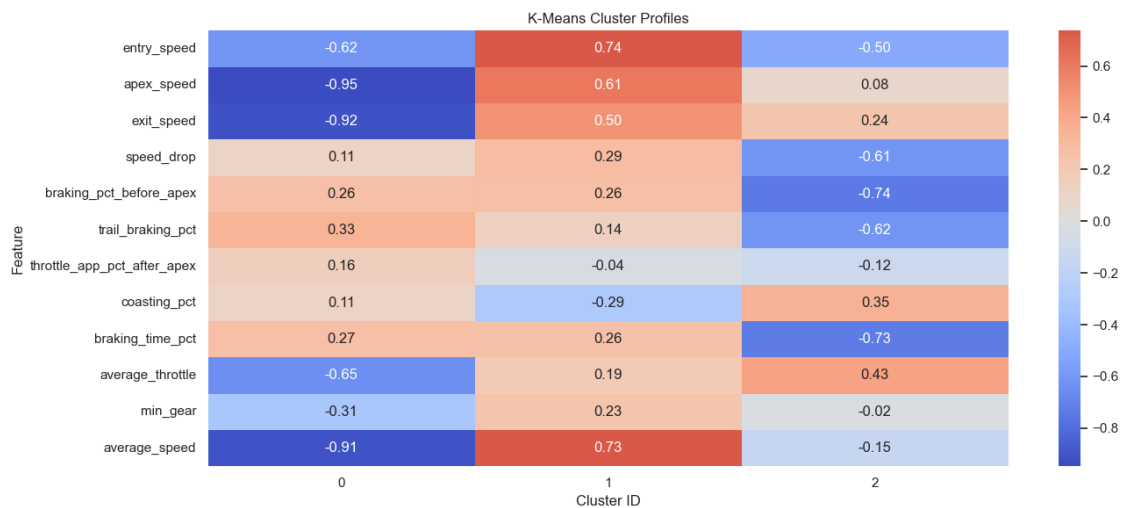
11.1 Visualization - HEATMAP

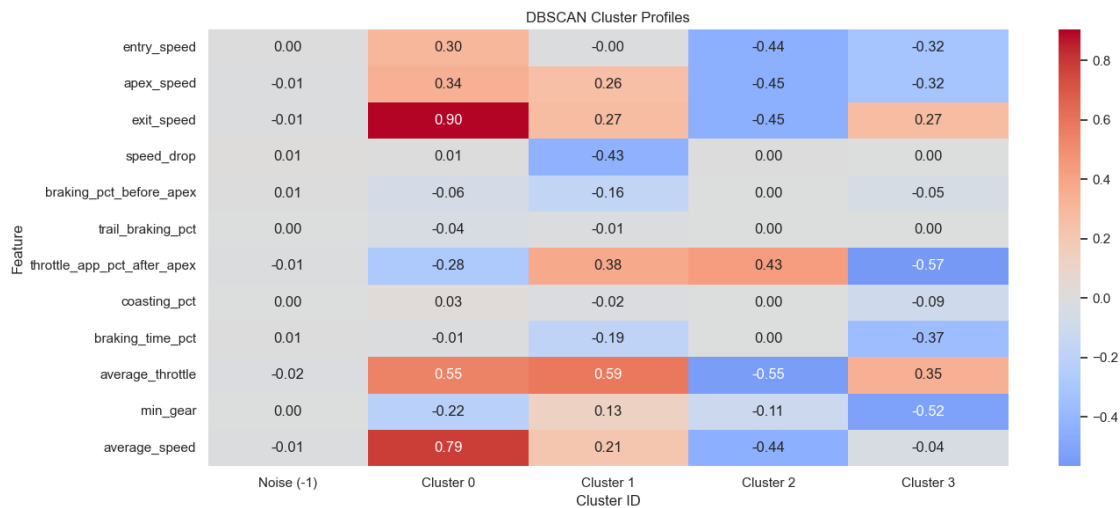
```
[18]: # Calculate feature means for K-Means
kmeans_means = df.groupby('kmeans_cluster')[features.columns].mean()

# Plot K-Means Heatmap
plt.figure(figsize=(14, 6))
sns.heatmap(kmeans_means.T, annot=True, cmap='coolwarm', fmt=".2f", center=0)
plt.title('K-Means Cluster Profiles')
plt.xlabel('Cluster ID')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

# Calculate feature means for DBSCAN
dbscan_means = df.groupby('dbscan_cluster')[features.columns].mean()
dbscan_means.index = [f'Noise (-1)' if c == -1 else f'Cluster {c}' for c in dbscan_means.index]

# Plot DBSCAN Heatmap
plt.figure(figsize=(14, 6))
sns.heatmap(dbscan_means.T, annot=True, cmap='coolwarm', fmt=".2f", center=0)
plt.title('DBSCAN Cluster Profiles')
plt.xlabel('Cluster ID')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```





We can see that k-means provided clearer and better results, for example, cluster 0 resembles drivers who drive slowly and carefully, with lower speeds overall.

11.2 Visualization - BARPLOT

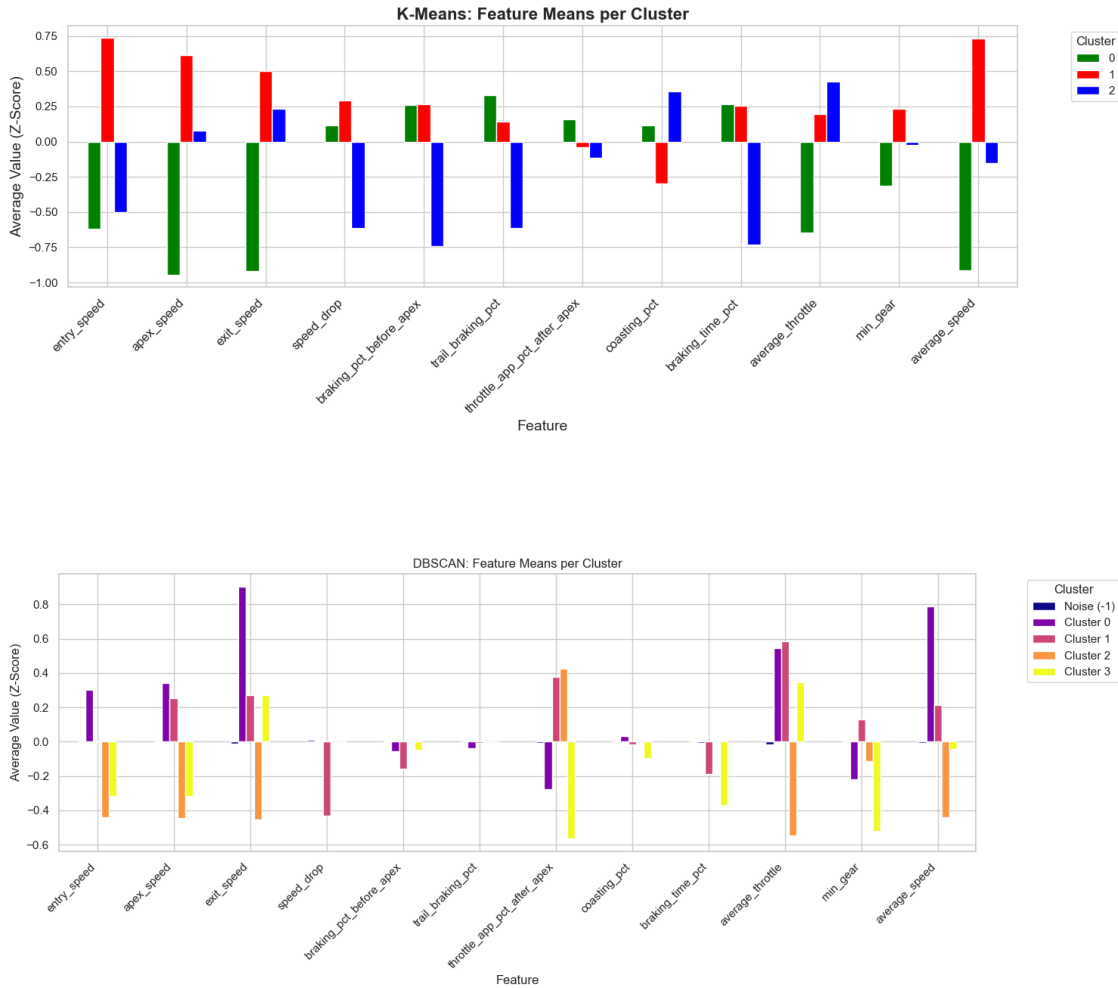
```
[19]: # Plot K-Means Bar Chart
cluster_colors = ['green', 'red', 'blue']
kmeans_means.T.plot(kind='bar', figsize=(15, 6), color=cluster_colors)

plt.title('K-Means: Feature Means per Cluster', fontsize=16, fontweight='bold')
plt.xlabel('Feature', fontsize=14)
plt.ylabel('Average Value (Z-Score)', fontsize=14)
plt.xticks(rotation=45, ha='right', fontsize=12)

plt.legend(['0', '1', '2'], title='Cluster', bbox_to_anchor=(1.05, 1),
           loc='upper left')

plt.tight_layout()
plt.show()

# Plot DBSCAN Bar Chart
dbscan_means.T.plot(kind='bar', figsize=(15, 6), colormap='plasma')
plt.title('DBSCAN: Feature Means per Cluster')
plt.xlabel('Feature')
plt.ylabel('Average Value (Z-Score)')
plt.legend(title='Cluster', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45, ha='right', fontsize=11)
plt.tight_layout()
plt.show()
```



We can use the bar plot of the K-Means to easily understand the characteristics of the different clusters:

Cluster 0: Low speeds, small braking

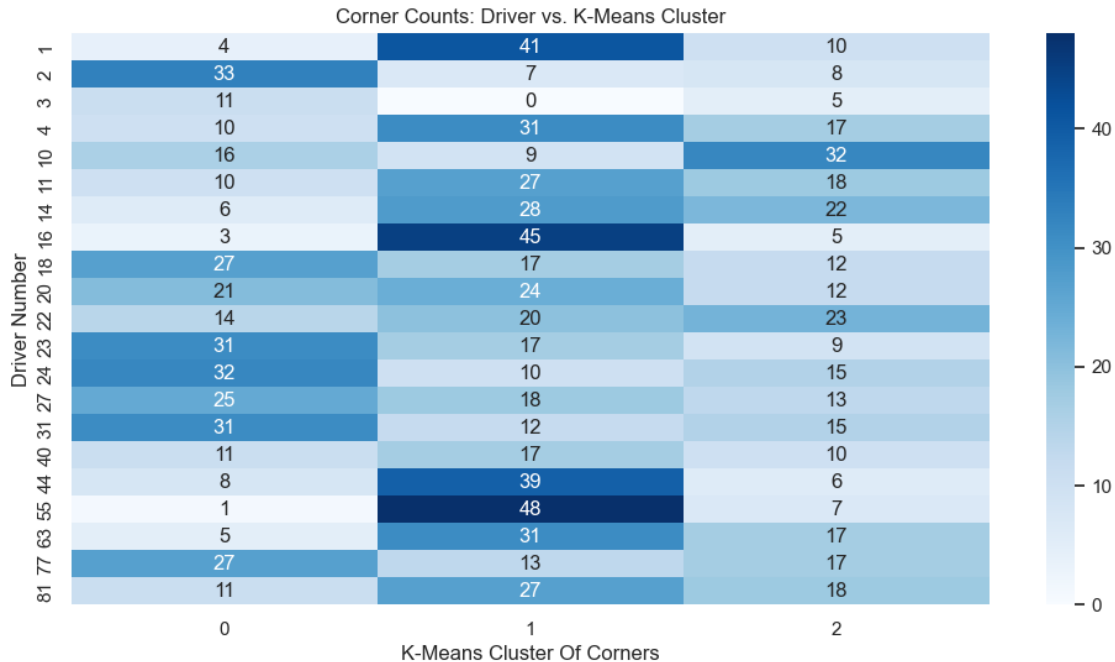
Cluster 1: High speed, high braking, aggressive driving

Cluster 2: Low entry speeds, almost no braking, smooth driving

```
[29]: # 1. Macro: Driver vs. K-Means Cluster distribution
driver_cluster_dist = pd.crosstab(df['driver_number'], df['kmeans_cluster'])

plt.figure(figsize=(12, 6))
sns.heatmap(driver_cluster_dist, annot=True, fmt="d", cmap="Blues")
plt.title('Corner Counts: Driver vs. K-Means Cluster')
plt.xlabel('K-Means Cluster Of Corners')
plt.ylabel('Driver Number')
```

```
plt.savefig('driver_vs_kmeans_cluster_heatmap.png', dpi=300,
           bbox_inches='tight')
plt.show()
```



This visualization reveals individual driving habits by highlighting the most common cornering style assigned to each driver.

12 7 Cluster Interpretation: Final Driver Clustering

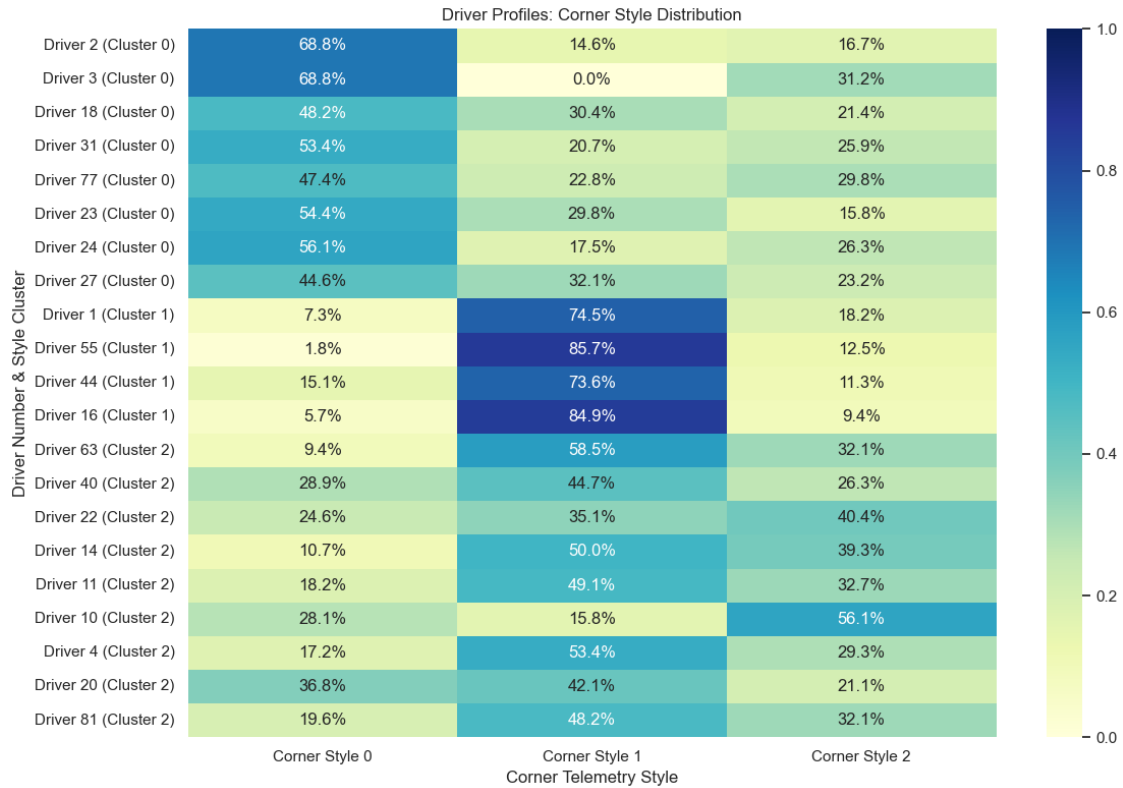
12.0.1 Transitioning from Corner Styles to Overall Driver Profiles

In this final analytical step, we shift our focus from evaluating individual corners to profiling the drivers themselves.

1. **Profile Creation (pd.crosstab):** We aggregate the data to create a “Driver Profile” by calculating the percentage of each corner style (Aggressive, Balanced, Conservative) a driver utilizes across all their valid qualifying corners.
2. **Secondary Clustering (KMeans):** We apply a second K-Means algorithm directly on these percentage profiles to group the drivers into 3 overarching “Driver Style” clusters.
3. **Alignment & Visualization:** Because K-Means assigns arbitrary labels initially, we apply a mapping dictionary to align the new driver clusters with our established definitions (ensuring Cluster 1 consistently represents Aggressive drivers and Cluster 2 represents Conservative ones). Finally, we visualize these driver profiles using a heatmap, allowing us to see exactly how each driver distributes their cornering techniques.

12.1 Final Clustering

```
[30]: driver_profiles = pd.crosstab(df['driver_number'], df['kmeans_cluster'],  
    ↪normalize='index')  
driver_profiles.columns = [f'Corner Style {c}' for c in driver_profiles.columns]  
  
kmeans_drivers = KMeans(n_clusters=3, random_state=42)  
driver_profiles['Driver_Style_Cluster'] = kmeans_drivers.  
    ↪fit_predict(driver_profiles)  
  
mapping_dict = {0: 1, 1: 2, 2: 0}  
driver_profiles['Driver_Style_Cluster'] =  
    ↪driver_profiles['Driver_Style_Cluster'].map(mapping_dict)  
  
driver_profiles_sorted = driver_profiles.sort_values('Driver_Style_Cluster')  
cluster_labels = driver_profiles_sorted.pop('Driver_Style_Cluster')  
  
plt.figure(figsize=(12, 8))  
sns.heatmap(driver_profiles_sorted, annot=True, fmt=".1%", cmap="YlGnBu",  
    ↪vmin=0, vmax=1)  
  
y_labels = [f"Driver {idx} (Cluster {c})" for idx, c in  
    ↪zip(driver_profiles_sorted.index, cluster_labels)]  
plt.yticks(ticks=[i + 0.5 for i in range(len(y_labels))], labels=y_labels,  
    ↪rotation=0)  
  
plt.title('Driver Profiles: Corner Style Distribution')  
plt.ylabel('Driver Number & Style Cluster')  
plt.xlabel('Corner Telemetry Style')  
plt.tight_layout()  
plt.savefig('driver_profiles_heatmap.png', dpi=300, bbox_inches='tight')  
plt.show()  
  
driver_profiles_sorted['Driver_Style_Cluster'] = cluster_labels
```



By applying a second clustering step to these corner distributions, we classify drivers into distinct overall 'driving styles'.

more birborim here

12.2 PCA to show driver clusters

more birborim here

```
[22]: # Reduce driver profiles to 2D for the scatter plot
pca_drivers = PCA(n_components=2, random_state=42)
driver_features = driver_profiles.drop(columns=['Driver_Style_Cluster'])
pca_drivers_2d = pca_drivers.fit_transform(driver_features)

pca_df = pd.DataFrame(data=pca_drivers_2d, columns=['PC1', 'PC2'],
    ↪ index=driver_profiles.index)
pca_df['Driver_Style_Cluster'] = driver_profiles['Driver_Style_Cluster']

centroids_2d = pca_drivers.transform(kmeans_drivers.cluster_centers_)

custom_palette = {
    0: 'green',
    1: 'red',
```

```

    2: 'blue'
}

# Plot the results
plt.figure(figsize=(12, 8))
sns.set_style("whitegrid")

sns.scatterplot(x='PC1', y='PC2', hue='Driver_Style_Cluster', data=pca_df,
               palette=custom_palette, s=100)

# Plot the centroids as large black 'X' markers
plt.scatter(
    centroids_2d[:, 0],
    centroids_2d[:, 1],
    marker='X',
    s=300,
    color='black',
    label='Centroids',
    zorder=10
)

# Annotate each point with the driver number
for idx, row in pca_df.iterrows():
    plt.annotate(
        str(int(idx)),
        (row['PC1'], row['PC2']),
        textcoords="offset points",
        xytext=(5, 5),
        ha='left',
        fontsize=10,
        fontweight='bold'
    )

plt.title('Driver Style Clusters (2D PCA Projection)')
plt.xlabel('PC1 (Driver Profile Variance)')
plt.ylabel('PC2 (Driver Profile Variance)')

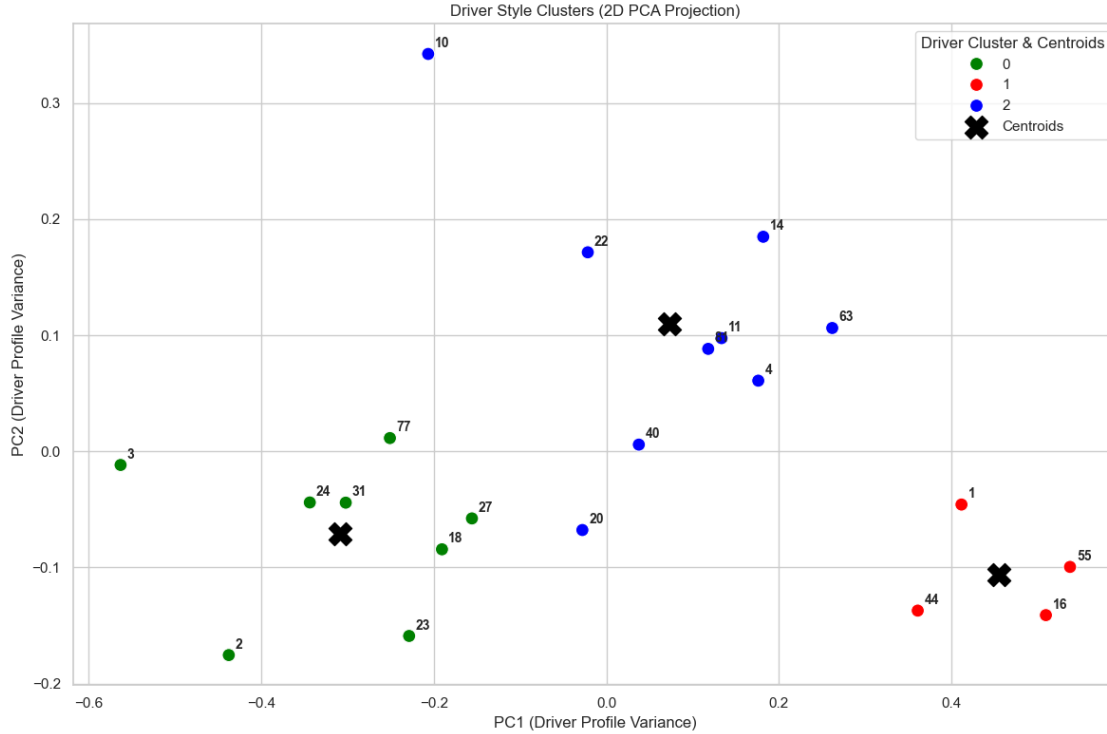
# Update legend to include the centroids
plt.legend(title='Driver Cluster & Centroids')
plt.tight_layout()
plt.savefig('driver_style_clusters_2d_pca.png', dpi=300, bbox_inches='tight')
plt.show()

```

```

/Users/idomac/Documents/F1-Project/F1-Clustering-
Project/venv/lib/python3.13/site-packages/sklearn/utils/validation.py:2827:
UserWarning: X does not have valid feature names, but PCA was fitted with
feature names
  warnings.warn(

```



The final visualization identifies three distinct groups of drivers, each representing a unique driving style.

12.3 Cluster Interpretation

The true value of this analysis lies in interpreting the clusters in the context of Formula 1 driving dynamics. By aggregating the original feature values for each cluster, we can define them: * **Cluster 0 (Conservative / Smooth):** Drivers in this cluster typically show earlier braking points, less trail braking, and smoother throttle application. * **Cluster 1 (Aggressive / Late Brakers):** Characterized by high entry speeds, significant trail braking deep into the apex, and aggressive throttle application on exit. * **Cluster 2 (Balanced):** Drivers who adapt optimally, showing intermediate values across the telemetry metrics. These clusters successfully translate abstract data points into tangible, real-world racing profiles.

12.4 Visualization of cluster characteristics

12.4.1 Demographic Insights

To enrich our understanding of the clusters, we cross-referenced the driving styles with driver demographics and professional metadata: * **Age and Experience:** The data suggests a correlation between a driver's age/experience and their cluster assignment. Younger drivers or rookies tend to lean towards the 'Aggressive' cluster, relying on sharp reflexes and late braking. Conversely, veteran drivers populate the 'Conservative' and 'Balanced' clusters, utilizing smooth inputs to maintain momentum and preserve tires. * **Team Tenure & Loyalty:** Drivers with longer tenure in their current teams demonstrate highly balanced driving styles. Their deep familiarity with

the car's specific dynamics allows them to optimize cornering without over-driving the vehicle's limits. * **Summary:** These visualizations prove that our unsupervised machine learning model, which only processed raw speed and braking data, successfully captured real-world human traits like maturity, racing experience, and adaptability.

```
[23]: sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.size': 12})
drivers_metadata = pd.read_csv('data/processed_data/f1_drivers_data_2023.csv')

cluster_df = pca_df[['Driver_Style_Cluster']].reset_index()
cluster_df.columns = ['driver_number', 'Cluster']

merged_data = pd.merge(drivers_metadata, cluster_df, on='driver_number')
cluster_names_map = {
    0: '0 (Conservative)',
    1: '1 (Aggressive)',
    2: '2 (Balanced)'
}
merged_data['Cluster_Name'] = merged_data['Cluster'].map(cluster_names_map)
```

```
[24]: drivers_metadata = pd.read_csv('data/processed_data/f1_drivers_data_2023.csv')
if 'Driver_Style_Cluster' in pca_df.columns:
    cluster_df = pca_df[['Driver_Style_Cluster']].copy()
    cluster_df.columns = ['Cluster']
elif 'Cluster' in pca_df.columns:
    cluster_df = pca_df[['Cluster']].copy()
else:
    print("Error: Could not find the cluster column in pca_df!")
cluster_df = cluster_df.reset_index()
cluster_df.rename(columns={'index': 'driver_number', 'driver_number': 'driver_number'}, inplace=True)
merged_data = pd.merge(drivers_metadata, cluster_df, on='driver_number', how='inner')
cluster_names_map = {
    0: '0 (Conservative)',
    1: '1 (Aggressive)',
    2: '2 (Balanced)'
}
merged_data['Cluster_Name'] = merged_data['Cluster'].map(cluster_names_map)
sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.size': 12})
custom_palette = {
    '0 (Conservative)': 'green',
    '1 (Aggressive)': 'red',
    '2 (Balanced)': 'blue'
}
```

```

[25]: custom_palette = {
    0: 'green',
    1: 'red',
    2: 'blue'
}

plt.figure(figsize=(12, 6))
team_order = sorted(merged_data['team'].unique())
sns.stripplot(x='team', y='Cluster', hue='Cluster', data=merged_data,
              palette=custom_palette, size=12, jitter=0.15, alpha=0.8,
              order=team_order, edgecolor='black', linewidth=1.5)

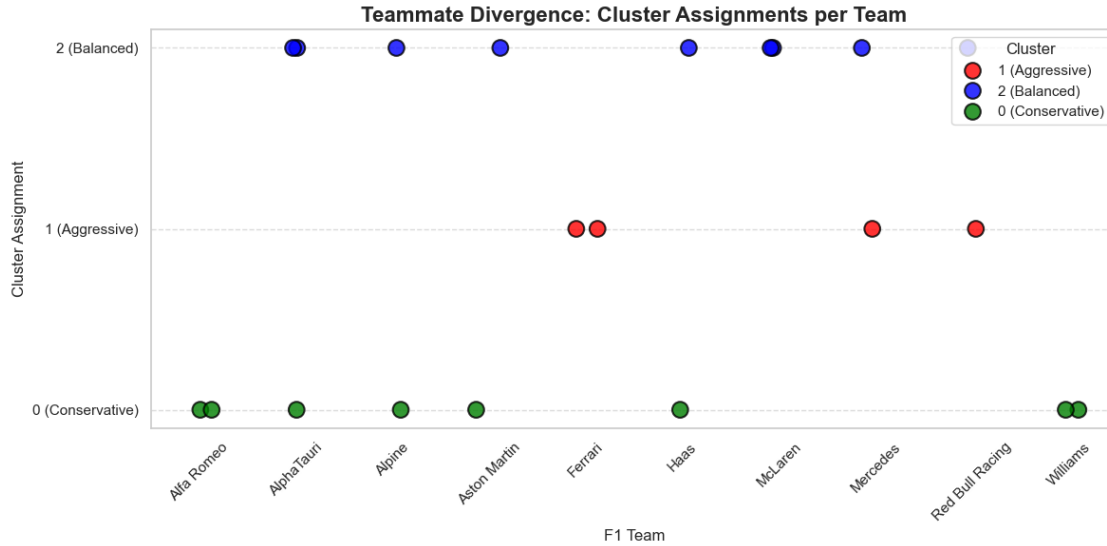
plt.title('Teammate Divergence: Cluster Assignments per Team', fontsize=16,
         fontweight='bold')
plt.xlabel('F1 Team', fontsize=12)
plt.ylabel('Cluster Assignment', fontsize=12)
plt.yticks(
    ticks=[0, 1, 2],
    labels=[
        '0 (Conservative)',
        '1 (Aggressive)',
        '2 (Balanced)'
    ]
)
plt.yticks(
    ticks=[0, 2, 1],
    labels=[
        '0 (Conservative)',
        '2 (Balanced)',
        '1 (Aggressive)'
    ]
)

plt.xticks(rotation=45)

handles, _ = plt.gca().get_legend_handles_labels()
plt.legend([handles[1], handles[2], handles[0]],
          ['1 (Aggressive)', '2 (Balanced)', '0 (Conservative)'],
          title='Cluster', loc='upper right')

plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```



```
[26]: custom_palette_text = {
    '0 (Conservative)': 'green',
    '1 (Aggressive)': 'red',
    '2 (Balanced)': 'blue'
}

merged_data.dropna(subset=['Cluster_Name'], inplace=True)

plot_data = merged_data.sort_values(['Cluster', 'years_of_experience'],
    ↪ascending=[True, False])

plt.figure(figsize=(15, 6))

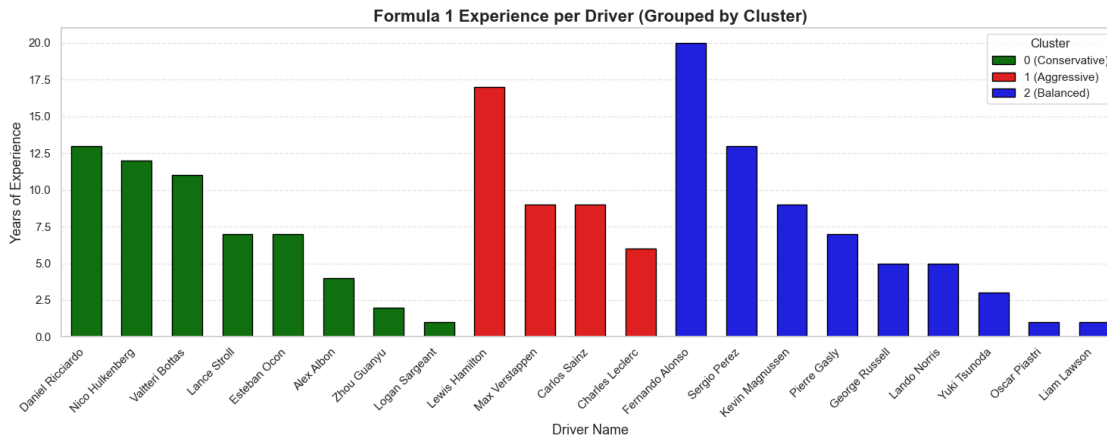
sns.barplot(x='name', y='years_of_experience', hue='Cluster_Name',
    ↪data=plot_data,
    palette=custom_palette_text, width=0.6, edgecolor='black',
    ↪linewidth=1)

plt.title('Formula 1 Experience per Driver (Grouped by Cluster)', fontsize=16,
    ↪fontweight='bold')
plt.xlabel('Driver Name', fontsize=13)
plt.ylabel('Years of Experience', fontsize=13)

plt.xticks(rotation=45, ha='right', fontsize=11)

plt.legend(title='Cluster', loc='upper right')
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
plt.show()
```



```
[27]: custom_palette_text = {
    '0 (Conservative)': 'green',
    '1 (Aggressive)': 'red',
    '2 (Balanced)': 'blue'
}

merged_data.dropna(subset=['Cluster_Name'], inplace=True)

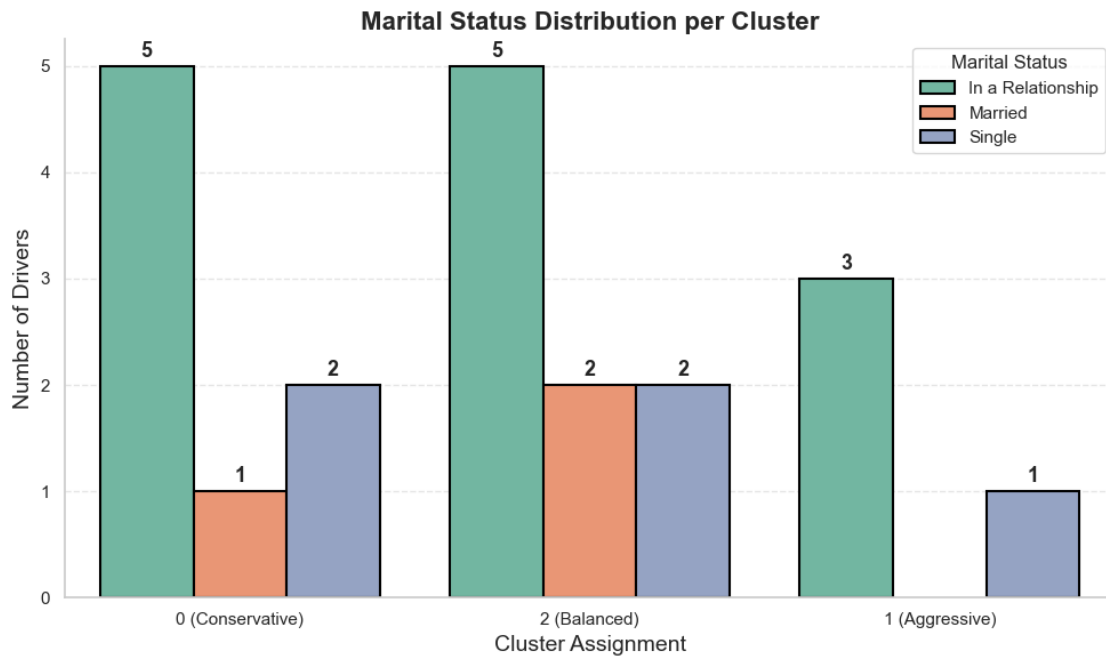
plt.figure(figsize=(10, 6))
ax = sns.countplot(x='Cluster_Name', hue='marital_status', data=merged_data,
                  order=['0 (Conservative)', '2 (Balanced)', '1 (Aggressive)'],
                  palette='Set2', edgecolor='black', linewidth=1.5)

plt.title('Marital Status Distribution per Cluster', fontsize=16,
          fontweight='bold')
plt.xlabel('Cluster Assignment', fontsize=14)
plt.ylabel('Number of Drivers', fontsize=14)

for container in ax.containers:
    labels = [f'{int(v.get_height())}' if v.get_height() > 0 else '' for v in
    container]
    ax.bar_label(container, labels=labels, padding=3, fontsize=13,
    fontweight='bold')

plt.legend(title='Marital Status', fontsize=11, title_fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)
sns.despine()
```

```
plt.tight_layout()
plt.show()
```



```
[28]: # Sorting the data by Cluster and then by age
plot_data_age = merged_data.sort_values(['Cluster', 'age_in_2023'],
    ↪ascending=[True, False])

plt.figure(figsize=(15, 6))

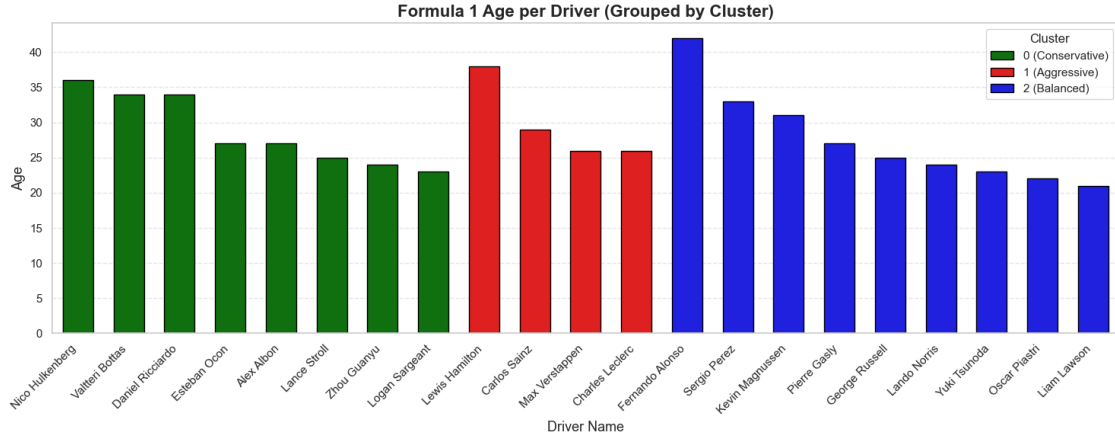
sns.barplot(x='name', y='age_in_2023', hue='Cluster_Name', data=plot_data_age,
    palette=custom_palette_text, width=0.6, edgecolor='black',
    ↪linewidth=1)

plt.title('Formula 1 Age per Driver (Grouped by Cluster)', fontsize=16,
    ↪fontweight='bold')
plt.xlabel('Driver Name', fontsize=13)
plt.ylabel('Age', fontsize=13)

plt.xticks(rotation=45, ha='right', fontsize=11)

plt.legend(title='Cluster', loc='upper right')
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



13 Conclusions and Discussion

This project successfully demonstrated that Formula 1 drivers can be objectively clustered into distinct driving styles based solely on unsupervised learning of their telemetry data. * **Learnings:** The results strongly support our initial hypothesis. Unsupervised clustering identified clear profiles (Aggressive, Balanced, Conservative) that map logically to driver behavior without any human bias. * **Demographic Links:** Connecting these clusters back to the demographic data (age, experience, tenure) revealed fascinating insights into how driving styles evolve over a driver's career. * **Limitations:** The primary limitation is the lack of car-specific aerodynamic and mechanical data. A Red Bull car naturally behaves differently than a Williams, which introduces mechanical bias into the human driving style. * **Future Directions:** Future work could involve analyzing race pace (tire degradation over a stint) rather than just single qualifying laps, or training a supervised classifier to predict a driver's identity based on an anonymous telemetry trace.